
KISS Sorcar: A Stupidly-Simple General-Purpose and Software Engineering AI Assistant

Koushik Sen

EECS Department, UC Berkeley
ksen@berkeley.edu

Abstract

Users ask coding agents to make a system faster or more robust and let them run for hours, past the context window, where one untested change can undo a day’s work. KISS Sorcar is an open-source general-purpose agent for such runs that doubles as an IDE. Its behavior is set by five Python agent classes of 4,531 significant lines, one concern per layer (budget-tracked ReAct, continuation from a written summary, tools with parallel sub-agents, persistent chat, a git worktree per development task), and by a 3,851-word system prompt that states engineering rules and two procedures: an AI discovery loop that logs tried ideas to files, and adversarial testing by alternating break and fix sub-tasks. One sentence in a task prompt adds a read-only reviewer from a second vendor. The agent’s task database records six months of it developing itself. A controlled ablation on 10 small coding tasks finds that the prompt’s rules change how the agent works and leave the hidden-test pass rate unchanged, and that the second-vendor reviewer finds real defects the tests missed. Two long runs built a key-value store that sustains 5.50 Mops/s on YCSB-A and made a TPC-H engine $34.5\times$ faster than its single-threaded artifact; each is one run without a baseline agent. On 30 Terminal-Bench 2.0 tasks and seven models, with a shorter coding prompt, the agent solves 75.6% of attempts against 70.0% for Pi, the best published harness, a gap within sampling error; against Pi rerun on one model, the paired gap is +11.1 points.

1 Introduction

Modern large language models (LLMs), such as Anthropic’s Claude Opus 4.7 [Anthropic, 2026b], OpenAI’s GPT 5.5 [OpenAI, 2026b], and Google’s Gemini 3.1 [Google DeepMind, 2026], can generate code, reason about software architecture, and use developer tools [Chen et al., 2021, Rozière et al., 2023]. A growing body of work has explored how to use these capabilities for autonomous software engineering, from single-session agents that resolve GitHub issues [Yang et al., 2024b, Wang et al., 2025b] to industrial products marketed as AI software and general assistants [GitHub, 2021, Anysphere, Inc., 2026, Wu, 2024, Anthropic, 2025b, OpenAI, 2025a, OpenClaw AI, 2025]. Yet using an LLM as a practical *general-purpose* assistant for long-horizon software engineering and AI-discovery tasks still exposes several gaps: context windows are finite, a single mistake can derail an entire session, agents get stuck in dead ends, models generate AI slop, and generated changes are difficult to review or revert once applied to a live codebase.

The gaps widen when a user asks the agent to make a system faster, or a model better, or a program correct on inputs it has not seen, and lets it run for hours through a loop of hypothesis, change, measurement, and keep-or-discard [Karpathy, 2026, Huntley, 2025]. The context window fills and what the agent tried is lost unless it was written down. An edit to a file the agent has not read, or a fix it never ran, breaks the artifact hours after the goal was last checked. A measured gain may be an overfit to the one workload the agent tuned on, and the model that made a mistake is the one asked to find it. FunSearch [Romera-Paredes et al., 2024], AlphaEvolve [Novikov et al., 2025], and the AI

Scientist [Lu et al., 2024] run such loops inside purpose-built systems with the loop in code. General coding agents are compared on benchmarks such as SWE-bench [Jimenez et al., 2024], whose tasks take minutes and whose leaderboard can no longer be ordered statistically [Liu et al., 2026], and Ma et al. [2026] find agents scoring near-perfectly against a hidden oracle while the library they were asked to build is unusable. We ask whether an ordinary coding agent can be brought to carry long runs with rules in a prompt and no loop written in code.

We propose the *KISS Agent Framework*, whose five core agent classes contain 4,531 significant lines of Python. We address the gaps above through a structured system prompt (Appendix C) and a five-layer agent hierarchy in which each layer solves one concern:

1. **KISS Agent:** budget-tracked ReAct [Yao et al., 2023b] loop with native function calling.
2. **Relentless Agent:** automatic summarization and continuation across sub-sessions.
3. **Sorcar Agent:** coding tools, browser automation, and parallel sub-agent execution.
4. **Chat Sorcar Agent:** persistent multi-turn chat sessions with history recall.
5. **Worktree Sorcar Agent:** git worktree isolation so that a development task runs on its own branch by default.

The name “KISS” reflects the *Keep It Simple, Stupid* design philosophy in software engineering: each layer is small, each concern is isolated, and the overall system avoids unnecessary abstraction.

What is and is not new. None of the five mechanisms is ours alone. Summary-and-restart continuation is the Ralph loop that Huntley [2025] published as one line of shell (`while :; do cat PROMPT.md | claude-code; done`) with a plan file on disk, and OpenHands ships an `LLMSummarizingCondenser` that keeps the head and tail of a conversation around a summary [OpenHands, 2026]; Claude Code compacts a full context automatically [Anthropic, 2026d]. Worktree isolation is available as `claude --worktree` in Claude Code [Anthropic, 2026f] and powers Cursor’s parallel agents [Cursor Team, 2025]. The prompt-as-driver loop of Appendix A is the shape of Karpathy [2026]’s `autoresearch`, a `program.md`-driven keep-or-discard experiment loop. A second model reviewing a first model’s diff is LLM-as-judge applied to code. The 32 messaging channels, the A2A agent, sender pairing, and the gateway tick follow the catalog of OpenClaw [OpenClaw AI, 2025] item for item; no OpenClaw code was used, and every channel agent was written in this repository by the agent itself. What is new is the assembly of these mechanisms in one small codebase; a worktree pool with crash reclaim (Section 2.5); one `run_agent` tool that dispatches to 42 channel and service agents; the AI discovery and adversarial-testing procedures stated in plain text in the system prompt (Section 2.6); a typed decision tool and a pre-run task classifier backed by a non-generative decisions model (Section 2.3); and the standing use of a read-only reviewer model from a second vendor, which we measure in Section 3.

Three sizes. The number that carries the word “simple” needs care. The five agent classes are 4,531 significant lines (no blank, comment-only, or docstring lines). They call into the `agents/sorcar` and `core` packages, which together are 26,751 significant lines in 63 files (46,616 non-blank); that is the code a task executes. The whole non-test package is 77,279 significant lines in 184 files (128,591 non-blank), of which 23,626 are the third-party agents package (the messaging-channel, service, and infrastructure agents with their shared helpers) and 17,939 the Python of the `kiss-web` daemon (the clients’ TypeScript and JavaScript are not counted). Our claim is about the first number: the agent loop, continuation, tools, chat, and worktree logic a reader must understand to change how the agent behaves fits in five files. The other two say what it costs to ship that loop as a product.

Table 1 compares KISS Sorcar with Claude Code and Cursor on the vendors’ own documentation as of 18 September 2026; Section 5 compares it with Claude Code, Codex CLI, and Pi on Terminal-Bench 2.0.

We implement KISS Sorcar, an open-source general-purpose AI agent for long-horizon tasks and AI discovery, which doubles as an integrated development environment (IDE) on top of the KISS Agent Framework (Figure 1). Its surfaces are a VS Code extension and a browser/mobile web app (both chat clients with file/folder @-mentions), plus a Python client API (`kiss.server.sorcar.run`) for scripted one-shot tasks; all connect to the same local daemon (`kiss-web`). A `sorcar` terminal command additionally runs a single task in the current directory without the daemon. KISS Sorcar has browser support (open-source Chromium driven via Playwright), voice and multimodal support, Docker container support, MCP server discovery from `~/kiss/mcp.json` and project-local configuration files, and a library of Agent Skills loaded from `~/kiss/skills` and project-local directories.

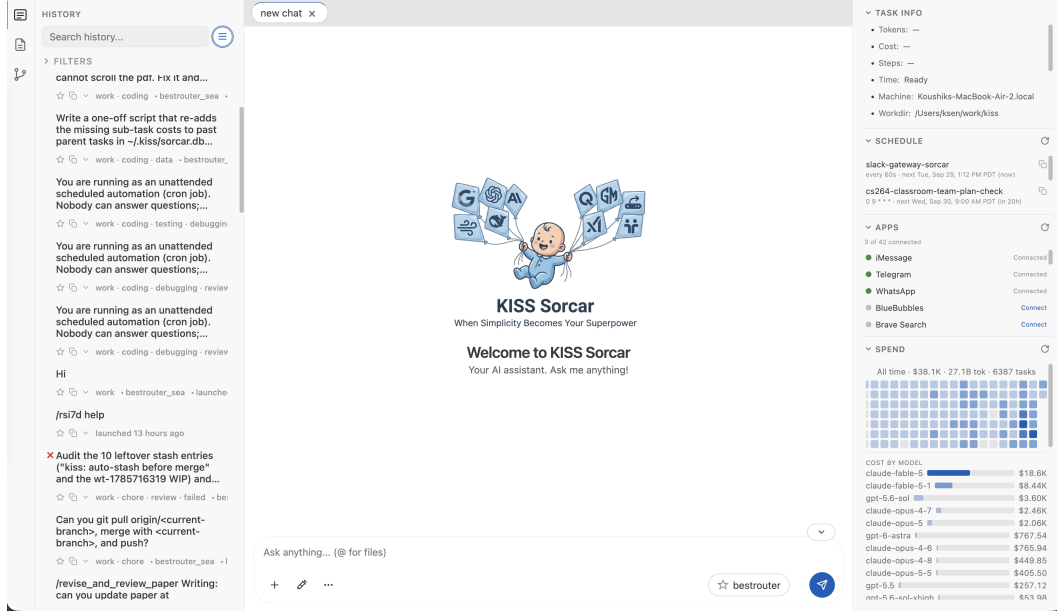


Figure 1: Screenshot of the KISS Sorcar chat client (the same page serves the VS Code extension and the browser/mobile web app). Left: the searchable task history, with the classifier labels and the slash command of each classified task. Center: the chat tabs, the welcome screen, and the prompt box with the selected model or router. Right: the Task Info view (tokens, cost, steps, machine, working directory), the Schedule list of cron jobs, the Apps list with the connection status of each service, and the Spend panel with a calendar heatmap of daily cost and a per-model breakdown.

KISS Sorcar is free and open-source (Apache-2.0), distributed on PyPI as `kiss-agent-framework`; all one needs is a model API key from a major LLM provider (Anthropic, OpenAI, Google, Together, Z.AI, Moonshot AI, or OpenRouter) or a custom local endpoint configured in the Settings panel. The framework can also run on top of the Claude Code CLI (`cc/*` models) and the Codex CLI (`codex/*` models). The repository is at https://github.com/ksenxx/kiss_ai; the task database extracts, the ablation harness, and both case-study artifacts are in it under `papers/kissorcar/` and `projects/`. The name “Sorcar” pays homage to P. C. Sorcar, the Bengali magician.

KISS Sorcar has been built using itself. The entire codebase (the KISS Agent framework, the Sorcar agent layers, the VS Code extension, and the system prompt) was developed by KISS Sorcar operating on its own repository, and the task database it keeps (`sorcar.db`) records the whole process: 23,438 tasks in 2,324 chats across 165 released versions between April and September 2026, 96% of the top-level tasks on this repository (Section 3.1). If the agent introduces a bug that impairs its ability to function, we ask the agent to fix it by analyzing the trajectory and code. The five core agent classes have stayed compact through that history: at version 2026.9.25 the KISS Agent module is 813 significant lines, the Relentless Agent 802, the Sorcar Agent 1,522, the Chat Sorcar Agent 422, and the Worktree Sorcar Agent 972. The Sorcar and Chat layers have over time absorbed Docker-aware tool variants, parallel sub-agent orchestration, dynamic model switching, MCP and skill integration, a persistent memory, a typed-decision tool, pre-run task classification, dispatch to third-party channel agents and to extension agents typed as slash commands, and worktree concurrency safety, within the layered, single-concern design.

In the project, we prioritize output quality over speed. In our experience, using a weaker or cheaper model often forces the developer to discard the agent’s work and retry, which raises the total cost of the task; the testing instructions of Appendix C therefore make the agent run linters, type checkers, and tests before it declares success. Section 3 measures what those instructions do and do not buy.

Contributions. (1) A five-layer agent framework whose behavior-defining code is 4,531 lines, with a system prompt that encodes engineering rules and, in the same plain text, an AI discovery loop and adversarial testing (Section 2 and Appendix C). (2) Evidence from six months of self-hosted use, mined from the agent’s own task database: continuation, steering, worktree, edit-discipline,

Table 1: KISS Sorcar compared with Claude Code and Cursor. Every Claude Code cell is taken from the Claude Code documentation [Anthropic, 2026f,d] and every Cursor cell from Cursor’s documentation and release notes [Cursor Team, 2025, Cursor, 2026c,b], all checked on 18 September 2026, except the skills and slash-command cells [Anthropic, 2026e, Cursor, 2026d], checked on 27 September 2026; KISS Sorcar cells are for version 2026.9.25. Source size is stated three ways for KISS Sorcar; the other two products do not publish source.

Capability	KISS Sorcar	Claude Code	Cursor
Interfaces	VS Code extension, web/mobile app, Python API, terminal command	CLI, VS Code and JetBrains extensions, desktop app, web, mobile, Chrome extension, Slack	Custom VS Code fork, CLI, web and mobile cloud agents, Slack
Isolation of a task’s edits	Every development task on its own branch and worktree by default, with a spare-worktree pool and crash reclaim	<code>cclaude --worktree <name></code> creates a worktree per session on request	Parallel agents run in git worktrees or on remote machines (Cursor 2.0)
Continuation past the context window	Structured summary, then a fresh sub-session	Automatic compaction into a summary within the session	<code>/goal</code> keeps a long-lived objective across turns; Projects split a large task across agents
Models	689 catalog entries from 9 provider categories; the agent may switch vendors mid-task with <code>set_model</code>	Anthropic models (direct or through Bedrock, Vertex, Foundry gateways)	User-selected model per request from a vendor mix chosen by Cursor, or Auto
Bring your own key / endpoint	Keys and prompts go from the user’s machine to the provider; any OpenAI-compatible endpoint	Anthropic key or gateway	API keys accepted; requests routed through Cursor’s backend
Scheduled and event-driven runs	Natural-language cron agent; inbound channel gateways	Routines (cloud, also on API calls and GitHub events), desktop scheduled tasks, GitHub Actions, <code>/loop</code>	Automations on cron and repository events
Messaging and service agents	45 bundled agents: 32 channels (Slack, Gmail, WhatsApp, SMS, phone control, ...), 10 service agents (GitHub, Notion, PostgreSQL, Google Workspace, ...), an agent that answers questions about a running task, 2 infrastructure agents	Slack, Remote Control from mobile, research-preview Telegram, Discord, iMessage	Slack and Microsoft Teams cloud-agent integrations
Shipped single-prompt drivers	Sample prompts for AI discovery, GEPA, and repository optimization; adversarial testing as a procedure in the system prompt (Appendix A); 14 bundled extension agents as slash commands (Appendix B.6)	Prompt library of everyday recipes; skills invoked as <code>/skill-name</code> , custom commands merged into skills	Skills in <code>.cursor/skills/</code> , invoked as <code>/skill-name</code> ; slash commands migrating to skills
Runs on top of Claude Code / Codex CLI	<code>cc/*</code> and <code>codex/*</code> model namespaces	N/A	No
Source	Apache-2.0; 4,531 lines (five agent classes), 26,751 (agents and core), 77,279 (package)	Proprietary	Proprietary
Price	Free; the user pays the model provider	Subscription or API usage	Subscription

test-discipline, and review statistics (Section 3.1). (3) A controlled ablation of the system prompt on 10 self-contained coding tasks with hidden tests, 6 arms and 2 repetitions each, and a controlled comparison of the cross-vendor read-only reviewer against its absence, with every reviewer finding re-checked against the other arms’ artifacts (Sections 3.2 and 3.3). (4) An open record of two long runs carried out from prompts with numeric goals, HydraKV under an earlier one-paragraph discovery instruction and the Bespoke OLAP engine under the seven-step loop, released with their tests, logs, audits, and process records (Section 4); each is a single run with no rerun and no ablation of the procedure. (5) A run of the same agent loop and tools on the HarnessTax study’s 30 Terminal-Bench 2.0 tasks and seven models next to the study’s published numbers, a run on the 59 tasks the study did not sample, and a same-model run of Pi on the same tasks for a paired comparison (Section 5). The user-facing features, the single-prompt drivers, the full system-prompt commentary, and two development sessions are in the appendices.

Outline. Section 2 presents the five-layer architecture and the two procedures of the system prompt. Section 3 reports the task-database evidence, the ablation, and the reviewer experiment. Section 4 reports the HydraKV and Bespoke OLAP case studies, and Section 5 the Terminal-Bench 2.0 runs. Section 6 discusses related work, Section 7 the limitations, and Section 8 concludes. Appendix A

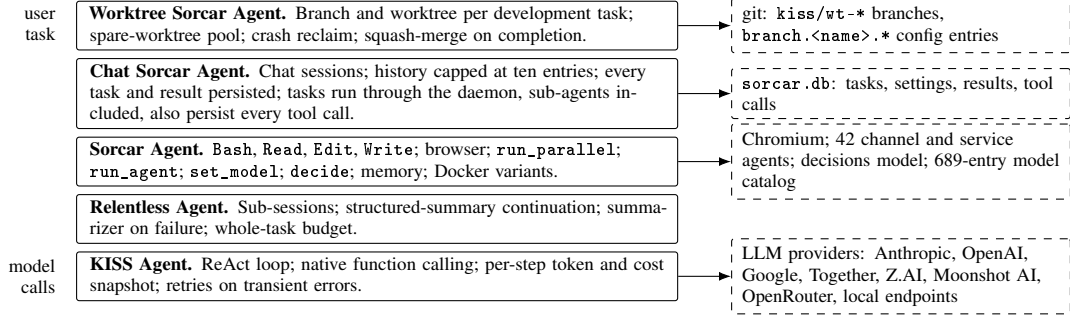


Figure 2: The five layers of the KISS Agent Framework and the external state each one touches. Each layer wraps the one below it, from the worktree that holds a chat down to the KISS Agents a Relentless Agent runs until the task is done.

describes the single-prompt drivers for AI discovery, adversarial testing, GEPA prompt optimization, and repository optimization; Appendix B the user-facing features; Appendix C the system prompt’s coding and testing instructions; Appendix D two development sessions; Appendix E the ablation tasks and arms; and Appendix F the case-study prompts and process record.

2 Agent Architecture

We initially built the KISS Agent Framework, and kept it small, to prototype prompt optimization techniques such as GEPA [Agrawal et al., 2026] and evolutionary algorithms for algorithmic and code optimization such as AlphaEvolve [Novikov et al., 2025] and OpenEvolve [Algorithmic Superintelligence, 2025]. We did not use any prompt optimization techniques when creating the system prompt for KISS Sorcar, as we hand-tuned it based on our long-term experience with KISS Sorcar and its behavior. The KISS Agent Framework uses five agent layers in a layered architecture combining composition and inheritance (Figure 2). Each layer delegates upward for the concerns it does not own.

2.1 KISS Agent

The KISS Agent is the innermost execution unit. It implements a standard ReAct loop [Yao et al., 2023b] with the following characteristics.

```
from kiss.core.kiss_agent import KISSAgent

def calculate(expression: str) -> str:
    """Evaluate a math expression."""
    return str(eval(expression))

agent = KISSAgent(name="Math Buddy")
result = agent.run(
    model_name="gemini-2.5-flash",
    prompt_template="Calculate: {question}",
    arguments={"question": "What is 15% of 847?"},
    tools=[calculate]
)
print(result)  # 127.05
```

Listing 1: A complete KISS agent with a single tool.

Native function calling. We register tools as ordinary Python callables. The agent builds an OpenAI-compatible tool schema once at setup time and caches it, avoiding redundant schema construction on every LLM call. A special `finish` tool signals task completion and returns the result to the caller.

Step, token, and budget tracking. At every model response, the agent extracts input and output token counts from the API response, computes the dollar cost from a per-model pricing table, and

publishes cost, tokens, and step count together as one immutable usage snapshot, so a concurrent reader (a live usage monitor, or the enclosing Relentless Agent recovering from an interrupted session) never sees the tokens of a response without its cost. Before each step of the ReAct loop the agent checks its own budget and calls an optional hook through which the enclosing Relentless Agent enforces the whole task’s cumulative budget; the loop itself enforces the maximum step count. (A run-to-completion CLI backend such as `cc/*` or `codex/*` receives the remaining task in one shot and is accounted for when it returns.)

Error resilience. The agent retries transient API errors (rate limits, server errors) and gives up after three consecutive failures (`MAX_CONSECUTIVE_ERRORS`). On a non-retryable error (authentication, a gated model, exhausted credit; an unknown model name fails before the loop) it switches once to the catalog’s fallback for the current model, an OpenRouter twin where one exists, and raises only when there is none.

Non-agentic mode. When tools are not needed, the agent can run a single generation without the ReAct loop, which is useful for summarization or question-answering sub-tasks.

Listing 1 shows a complete, working agent in a dozen lines of code. The developer defines an ordinary Python function (`calculate`), instantiates a `KISSAgent`, and calls its `run` method with a model name, a prompt template, template arguments, and a list of tools. The framework automatically handles tool-schema generation, the ReAct loop, and budget tracking.

The KISS Agent is stateless across runs: each call to its `run` method resets the conversation, token counters, and tool registry. This makes it safe to reuse a single agent instance for multiple sequential tasks.

2.2 Relentless Agent

The Relentless Agent wraps a KISS Agent in a continuation loop. Its core contribution is the ability to execute tasks that exceed a single context window by breaking them into sub-sessions.

The KISS Agent trims its context only mechanically: once the context passes 100,000 tokens (and after every further 100,000), old large tool outputs become short stubs (`tool_output_compaction`, which can be switched off) if that removes at least 25% of the context and the session is below 80% of its hand-off size, since a compaction costs a prompt-cache miss; nothing is summarized in place. For the rest we adopt a continuation protocol: when a sub-session reaches 70% of the model’s context window (`CONTEXT_LIMIT_FRACTION`, at which the KISS Agent raises `ContextWindowExceededError`) or exhausts its step budget, the agent produces a *structured summary* of every action taken so far (chronological, with explanations and code snippets) and a fresh sub-session resumes from it. The shape is the Ralph loop [Huntley, 2025], which restarts the agent from a plan file on disk on every iteration, with the plan replaced by a summary the agent writes at the moment it decides to hand off; OpenHands’ `LLMSummarizingCondenser` [OpenHands, 2026] and Claude Code’s automatic compaction [Anthropic, 2026d] keep the session and summarize its middle instead. Reflexion [Shinn et al., 2023] also carries a verbal summary between attempts, to improve a retry; ours continues the work (Section 6). In the task database, 773 of 23,438 tasks used at least one continuation and one task used 119 (Section 3.1). A naïve instruction to “summarize the current context” produced poor continuations; requiring a *step-by-step chronological account with code snippets* improved coherence across sub-sessions. Summaries may grow unwieldy for multi-day tasks; those of our tasks, up to several hours long, stayed manageable, and an evaluation of summary scaling remains future work.

Continuation protocol. The `finish` tool exposed to the inner KISS Agent accepts a success flag, a continue flag, an HTML summary, and an optional suggested next task for the user. When the agent sets `is_continue=True`, the Relentless Agent starts a new sub-session with a fresh context window. The prompt for the new session includes the prior attempt summaries in chronological order, capped at 60,000 characters with the oldest summaries dropped first (and a note saying how many were omitted), and instructs the agent not to redo completed work. The continuation prompt template is:

```
# Task Progress (Continuation {continuation_number})

{progress_text}

# Continue
- Complete the rest of the task.
```

- ****DON'T**** redo completed work.
- If you have been retrying the same approach without progress, step back and rethink the strategy from scratch.

Forced continuation on failure. If a sub-session fails with a framework-level error after making progress (e.g., the step limit is hit before the agent calls finish, or the context reaches the 70% hand-off point), the Relentless Agent does not abort. It saves the full trajectory to a temporary file, spawns a separate summarizer agent to read the trajectory and produce a concise summary, and then uses that summary as the progress text for the next sub-session. Errors that continuation cannot fix (exceptions from outside the framework, or a failure at the very first step) terminate the task with the error as its result. The summarizer receives the following prompt:

```
# Summarizer

The executor's trajectory is saved at: {trajectory_path}

Read relevant portions of the file using your tools:
- Read the first ~50 lines to understand the task and system instructions.
- Read the last ~200 lines to see the most recent steps and outcomes.
- Do NOT read the entire file; it may be very large.

# Instructions
- Analyze the trajectory file.
- Return a precise chronologically-ordered list of things the agent did with the reason for doing that along with relevant code snippets.
- Format the summary as HTML (e.g. <ol>, <p>, <pre><code>), never Markdown.
- Call finish(result="detailed summary of work done so far, in HTML").
```

To make the agent self-continue before it exhausts its context window, the Relentless Agent appends the following block to every executor's system prompt (the finish tool's summary field is named `summary_in_html` because every client renders it as HTML):

```
# MOST IMPORTANT INSTRUCTIONS
- **If the task is not complete and you are at risk of running out of context length, you MUST call finish(success=False, is_continue=True, summary_in_html="precise chronologically-ordered list of things the agent did with the reason for doing that along with relevant code snippets, formatted as HTML (e.g. <ol>, <p>, <pre><code>), never Markdown")**
- The summary_in_html argument of finish MUST always be formatted as HTML.
- Work dir: {work_dir}
- Current process PID: {current_pid} - NEVER kill this process.
```

An earlier revision named an explicit step threshold at which the agent had to continue; it was replaced by the context-length condition above because the step budget is rarely the binding constraint, and the KISS Agent already reports its current step count and context usage in every tool result, which is what the model needs to judge when to hand off. The loop stops on its own after two consecutive sub-sessions with no progress (`MAX_ZERO_PROGRESS_SESSIONS`: no tool called but `finish`, or the previous summary returned unchanged) or after `max_sub_sessions` (10,000 by default).

2.3 Sorcar Agent

The Sorcar Agent adds the tools that make the system useful for software development and general-purpose automation.

Coding tools. Six coding tools are on by default: Bash, a shell executor with streaming output that can also detach a command as a background job; `bash_job`, which waits for, tails, or kills such a job; `run_commands_parallel`, which runs independent shell commands in threads and returns every exit code and output in one report; Read; Edit, which replaces an exact string that must occur once

unless `replace_all` is set; and `Write`. The shell executor has a configurable timeout and a stop event through which the user cancels a running command. We kept the tool names (`Bash`, `Edit`, `Write`, `Read`) the same as in Claude Code, so the models see a tool vocabulary they already know. Since version 2026.9.19, `Edit` and `Write` refuse an existing file that the tool instance has neither shown the model through `Read` nor written itself (`Write` exempts scratch files under `tmp/` and the cron jobs' work directory), so the read-before-modify rule is enforced for those two tools (a `Bash` command can still change a file); the task database and the ablation of Section 3 predate this check.

Browser automation. A web-use tool provides programmatic browser control: navigating to URLs, reading page accessibility trees, clicking elements, typing text, pressing keys, scrolling, and taking screenshots. This enables the agent to research documentation, verify deployed applications, and interact with web-based tools. It uses the open-source Chromium browser using the Playwright library.

Parallel sub-agents. A parallel execution tool spawns one Chat Sorcar Agent (Section 2.4) per task string in a thread pool. Each sub-agent runs in its own LLM context, seeded with the parent chat's history, with its own tool set, and its tokens, cost, and steps fold into the parent task's accounting. This is useful for embarrassingly parallel tasks such as summarizing multiple files or researching independent topics. We collect the results and return them in input order.

User interaction. An ask-user-question tool allows the agent to pause execution and request clarification from the user. In the VS Code integration, this renders as a text input in the sidebar; the browser/mobile web app exposes the same input.

Delegation to agents. A `run_agent(task, agent)` tool dispatches a sub-task to any of the 42 dispatchable third-party agents of Appendix B.6 (the 32 messaging channels and 10 service agents; the agent that answers `/ask` questions about a running task and the two infrastructure agents are not dispatch targets) by name ("`slack`", "`gmail`", "`github`", ...), to any agent-script Python file (a Sorcar Extension Agent, Appendix B.5), or to the cron agent for scheduled automations; with no agent named, it runs a plain Sorcar sub-agent. The sub-task's tokens, cost, and steps fold into the calling task's accounting. Every such script is also a chat command: a prompt that starts with `/xxx` makes the session call `run_agent` with the script `xxx/xxx_sea.py` and the rest of the prompt as the task (Appendix B.6). The system prompt instructs the agent to call `run_agent` immediately for any task that acts on a messaging service, mailbox, or device, without first exploring the channel's source code.

Persistent memory. A general memory and one domain memory per repository the agent works in, each a directory of short Markdown pages with a semantic index under `~/.kiss/memories` (`kiss.core.memoryfield`; the repository's memory is the sub-directory named after its top-level directory, shared by the main checkout and its worktrees), are exposed through `memory_search`, `memory_pull`, `memory_read`, `memory_write`, `memory_list`, `memory_delete`, and `memory_refresh`. The accompanying protocol asks the agent to search memory before starting work, to record durable facts (root causes, conventions, user preferences) with their source paths or URLs so they can be re-verified, never to store secrets, and to delete pages that turn out to be wrong. Every search first runs an incremental index sync, so pages edited by a human or pulled from git are searchable without a separate reindex. The bundled `/git_extract_knowledge` agent (Appendix B.6) fills a repository's memory with curated pages about its code and history. A no-op summary tool, which the system prompt asks the agent to call every ten steps, lets the chat clients collapse the preceding event panels into a short recap.

Typed decisions. A `decide(state, questions)` tool sends a piece of text and a JSON object of typed questions to OpenRouter's decisions endpoint (`POST /api/alpha/decisions`, model `openrouter/~typesafe/jev-latest`) and returns calibrated probabilities from one non-generative call. Three question types are supported: *noul* (the probability that a statement about the text is true), *choice* (one of a fixed set of options, with per-option probabilities and a confidence), and *score* (a position on an ordered rubric, with per-level probabilities). The system prompt tells the agent to use it whenever it has to take a decision, because the call is faster and cheaper than asking the agent's own model to classify, route, or grade. Its cost and tokens fold into the running task's accounting, and the tool is registered only when an OpenRouter key is configured and the catalog lists the model, so the agent is never offered a tool that would fail on every call.

Pre-run task classification. Before a task starts, the agent classifies it (“Which kind of task is this?”: software development, git operations only, Internet search, simple, or ambiguous). By default one non-agentic call on the task’s own model returns the verdict as a bare JSON object, schema-enforced on Gemini and the OpenAI-compatible vendors and by prompt alone on Anthropic; the run-to-completion CLI models (`cc/*`, `codex/*`) are not classified by this route. With the “Classify with Jev” setting on (off by default since version 2026.9.25) and an OpenRouter key, the agent first asks the decisions model one typed *choice* question and falls back to the model call only if that fails. Two settings follow from the verdict. A task that involves neither software development nor Internet search runs on `SYSTEM_LITE.md`, a system prompt of 753 words that keeps a shortened identity block with the rule-precedence order, the visibility constraint, the rule for running long commands in the background, the voice-interaction rule, and most of the repository-specific block (channel dispatch, cron, artifacts, confirmation before irreversible actions) of the 3,851-word `SYSTEM.md` and drops the rest, so a question or a chat message does not carry the coding, testing, and research instructions on every step. A task that does not create or edit files (a question, a git merge, a message to send) runs without worktree isolation (Section 2.5); the verdict can only withhold a worktree that the user’s settings asked for, never add one that the caller disabled. Verdicts are memoized for seven days, keyed by the classification criteria, the task text, the model, and its configuration, so a re-submitted prompt launches without a classifier call and a change to the criteria retires every old verdict. On the 415 real and synthetic task prompts in `benchmarks/task_classifier/` (results generated 2026-09-20), the decisions classifier agreed with the hand labels on 89% (95% of the 344 prompts whose label the annotator marked certain) at a median of 0.25 s per task; the best of the four LLM classifiers measured, `claude-fable-5-1`, reached 84% at a median of 3.1 s and 230× the mean cost. Both classifiers are Settings-panel checkboxes; when classification is off, or when every applicable route fails, there is no verdict and the task runs as it would without a classifier.

Voice interaction. A talk tool synthesizes speech on every device that has a tab open for the running task, and users can speak to a running task; their words arrive as text input, and in the web app the wake words “Hey Sorcar” start the listening. The system prompt instructs the agent to reply in the language the user spoke.

Docker isolation. When a Docker image is specified, we replace the coding tools with five Docker-aware variants (`Bash` without background jobs, `run_commands_parallel`, `Read`, `Edit`, `Write`) that execute commands inside a container, a further sandbox for untrusted tasks. The task’s working directory is bind-mounted into the container at its host path and set as the container’s working directory, so relative paths agree inside and out; `container:<name>` attaches the tools to a container that is already running.

Dynamic model switching with `set_model`. A `set_model` tool, registered under the default full tool profile and the assistant profile (the reduced review, `shell`, and `bash` profiles omit it), lets the agent hand the live conversation off to a different LLM at any point during a task without restarting it. The tool accepts a model name (for example “`gpt-5.5`”, “`claude-opus-4-7`”, or “`gemini-2.5-flash`”), constructs the new backend through the same factory that built the original model, copies the conversation history into the new backend (the cumulative usage snapshot lives on the executor and is untouched by the switch), rebuilds the cached tools schema in the new provider’s dialect (Anthropic and OpenAI schemas differ in subtle ways), shows the new model in the picker of every tab watching the task, and returns a confirmation string back to the agent loop. The switch is deliberately display-only: it never overwrites the user’s own picker choice, which the tabs restore when the task ends. If no model has been instantiated yet (deferred-startup case), it simply updates the agent’s default `model_name` for the next run; if the requested model is already active, it is a no-op; and inside a Docker-isolated task it refuses to switch to a `cc/*` or `codex/*` CLI backend, because those run natively on the host and would bypass the container.

This tool enables *multi-model*, *multi-vendor workflows inside a single task*. Three idiomatic patterns recur in our usage: (i) a *scout-then-edit* pattern where a cheap fast model (e.g. `gemini-2.5-flash`) reads the repository, grep-searches for relevant call sites, and gathers context, then `set_model("claude-opus-4-7")` hands off to a stronger reasoner for the actual edit; (ii) a *generate-then-review* pattern where a primary model produces a change and the agent calls `set_model("gpt-5.5")` to switch to a second-opinion model that re-reads the diff, runs the tests, and verifies the change against the original request; and (iii) a *cost-aware long-loop* pattern where an open-ended optimization or discovery task (Appendix A) spends a cheap model on bookkeeping and journal-keeping but pays for a frontier model only at the decision points. Because the conversation

transcript is carried across the switch and the usage snapshot stays with the executor, later budget accounting and continuation summaries stay coherent however often the model changes.

The following two prompts, reproduced from `src/kiss/INJECTIONS.md`, illustrate what this tool layer makes possible. Each is a single natural-language instruction that can be pasted into the IDE or web app; none is an API call or a special mode. They work because dynamic model switching, coding tools, test execution, parallel sub-agents, and the later layers’ persistence and worktree isolation are all available within one task.

Primary model plus independent review.

```
Use 'claude-fable-5' model for all tasks, including software
development. Use 'gpt-5.6-sol' (not codex) using 'run_parallel'
tool for a thorough read-only review and debugging of the other
model's work. Thoroughly check whether the other model has
missed any code or wiring or introduced any bugs. Use at most
50% of the task budget in gpt-5.6-sol for reviewing and
debugging, and ask the model not to invent new problems. Use
the model names literally without hallucinating new model
names.
```

Open-weight generation, frontier review.

```
Use 'openrouter/moonshotai/kimi-k3' model for all tasks,
including software development. Use 'gpt-6-astra' (not codex)
using 'run_parallel' tool for a thorough read-only review and
debugging of the other model's work. [...]
```

Cost-aware model routing. An earlier promptlet asked for routing in prose (pick the cheapest adequate model per sub-task and record the strategy in a `ROUTING.md`). Since version 2026.9.25 it is a bundled extension agent instead: `autorouter` appears in the model picker beside the models, and a task run through it is split into units with a mechanical acceptance check, each unit classified with decide into a small, medium, or frontier tier, dispatched through `run_agent` to the first runnable model of that tier (the tiers are the `TIERS` table of `seas/autorouter/autorouter_sea.py`, ordered by coding quality per dollar), verified through the check, and escalated one tier on a verified failure, with every decision appended to `~/kiss/MODEL_DECISIONS.md`. A second picker entry, `bestrouter`, is the first promptlet above as an agent.

The review clauses ask for the reviewer to run through `run_parallel`, so the review is a separate sub-agent with its own context and no authorship stake in the diff; the budget cap bounds what the review may spend, and the “do not invent new problems” clause restricts the report to evidenced findings. The clauses are requests in a prompt; what withholds the editing tools is that a sub-agent whose task reads as a review gets the review tool profile by default. We ask for a second vendor because a model favors its own output [Panickssery et al., 2024] and does not correct its own reasoning without external feedback [Huang et al., 2024]. A user can ask one model to implement and test, a model from a different vendor to review the result, and route the units of a task to models of different prices without leaving the same conversation.

2.4 Chat Sorcar Agent

The Chat Sorcar Agent adds multi-turn conversation persistence.

Chat sessions. We assign each task to a chat session identified by a stable chat ID. The agent persists tasks and their results to a local database (`sorcar.db`). When a new task arrives within the same chat session, the agent loads prior tasks and results and prepends them to the prompt as numbered context entries, allowing the LLM to reference earlier work.

Bounded chat context. To prevent unbounded growth of the prompt as a chat session accumulates many tasks, the agent caps the number of in-context history entries at `MAX_TASKS=10`. When the cap is exceeded, it preserves the first two entries (which typically establish the user’s overall intent for the session) and the most recent entries, dropping the middle entries that are least likely to be referenced. Because the prefix is re-sent on every step, by default (`chat_history_digest`) only the newest two results are quoted in full; older entries shrink to a digest of about 600 characters of task and 300 of result, and the oldest go first when the prefix still exceeds 6,000 characters.

Session management. Three operations manage sessions: `new_chat()` clears the chat ID (the next `run()` allocates a fresh one); `resume_chat_by_id` sets it, and the history loads with the next

prompt; and `resume_from_task_id` seeds the next prompt from one task’s parent chain, for a tab opened from the history on that task.

Frequent task tracking. Each time a task is executed, the agent records the task description in a frequency table, which the daemon serves on request; the current chat clients do not display it, and the History group of the input bar’s completion list (Appendix B.6) is drawn from the task history instead.

Metadata persistence. After each task, the agent records the model, working directory, version, tokens, cost, and whether parallel sub-agents and worktree isolation were on.

2.5 Worktree Sorcar Agent

The Worktree Sorcar Agent is the outermost layer, the one users interact with in the VS Code extension and the web app, and its defining feature is git-worktree isolation. Claude Code offers the same isolation on request through `claude --worktree` [Anthropic, 2026f], and Cursor runs parallel agents in worktrees [Cursor Team, 2025]; what this layer adds is that every task the pre-run classifier marks as development gets a worktree by default (a question or a lookup runs in the user’s checkout), that a spare is kept ready, and that worktrees left by a crashed process are reclaimed from state stored in git itself. In the task database, 48% of top-level tasks ran in a worktree and 1,023 were auto-committed and merged on completion.

Branch-per-task. When a task starts, the agent creates a new git branch and a corresponding worktree directory under `.kiss-worktrees/`. The branch name is `kiss/wt-<epoch>-<random suffix>`; worktrees are deliberately not tied to the chat session. Because `git worktree add` performs a full checkout, which on a large repository dominates the delay before the agent’s first step, a background pool keeps one spare worktree per repository ready, prepared when the daemon starts and again when a task is submitted, before the classifier has ruled; the next task consumes the spare and hard-resets it onto the tip of its original branch. The worktree is the task’s working directory, and the Bash tool rejects a command that spells out the main checkout’s absolute path; a command can still reach it through a relative or symbolic path.

Dirty-state preservation. If the user’s main working tree has uncommitted changes, the agent copies them into the worktree and commits the copy there as a baseline commit, leaving the user’s own index and working tree as they were, so it starts from the state the user sees. During merge, the agent stashes the main checkout’s uncommitted edits, cherry-picks from the baseline commit so that only its own changes are replayed, and restores the stash.

Merge. With auto-commit on, a finished task is squash-merged onto its original branch under a commit message a fast model writes from the staged diff (a conventional-commit subject, then the task prompt and the task’s result, converted from HTML to Markdown); the diff read is bounded, so a task that adds gigabytes does not stall the daemon. A merge whose conflicts git cannot resolve is handed to the bundled merge agent (`seas/merge`), which runs as a sub-agent of the task, reads both sides of every conflict block, and stages a resolution. A merge is deferred while a task on another tab works in the main checkout with uncommitted changes to tracked files, and is retried when that tree is clean again.

Concurrency safety. A per-repository in-process lock and a cross-process file lock shared with the reclaim pass serialize the stash, checkout, merge, and pop sequence, so concurrent tabs cannot interleave operations on one repository. Per-task state in the daemon’s printer (recordings, bash output buffers, subscribers) is keyed by task id, and per-thread state (the current block type, the thread’s task binding, stop event, and replay slot) is thread-local, so stopping one task does not corrupt another.

Crash recovery. We store all worktree state in git itself (the original branch, the baseline commit, and the owning process id as `branch.<name>.* config` entries), so there are no sidecar files to lose. A reclaim pass, run whenever the worktree pool refills and inline when a task finds no spare ready, finds `kiss/wt-*` worktrees whose owner process is no longer alive (killed, crashed, out of memory, or rebooted while a task was pending), auto-commits and squash-merges the ones that pass its safety guards so no dirty work is lost, and deletes the debris (registered worktrees whose directory is gone, orphaned branches, stale config sections) left by a crashed or killed process.

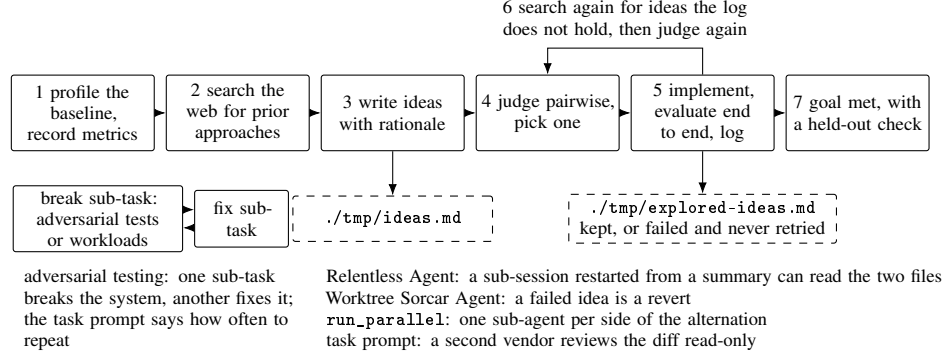


Figure 3: The two procedures the current system prompt states in prose (Appendix A); of the case studies, only the Bespoke OLAP task ran the seven steps, in an earlier wording. Top: the seven steps of the discovery and optimization loop with the two files that hold its state. Bottom: the break and fix alternation of adversarial testing, and the layer or tool that carries each step across a run of hours.

Graceful fallback. If the working directory is not inside a git repository, if the repository has no commits, or if HEAD is detached, the agent falls back to direct execution without worktree isolation, ensuring it never fails due to git preconditions.

2.6 Discovery and Adversarial Testing by Instruction

Two blocks of SYSTEM.md say how to run the long tasks of Section 4; Figure 3 draws them, Appendix F quotes the discovery block, Appendix A quotes the adversarial-testing block, and Algorithm 1 gives the loop as pseudocode. The first, headed “AI discovery, auto research, and optimization,” is seven numbered steps: profile the baseline; search the web for prior approaches; write candidate ideas with their rationale to `./tmp/ideas.md`; judge them pairwise to pick one; implement it, evaluate end to end, and log the idea, the aspect it improves, and its metrics to `./tmp/explored-ideas.md`, keeping it if better and trying to compose it with prior winners on other aspects, and marking it failed if worse so that it is never retried; search again for fresh ideas the log does not hold; and stop when the user’s metric goal is met, with a held-out or generalization check to show the result is not overfit. The second, “Adversarial testing,” is two sentences: use a sub-task to break the system with adversarial tests or workloads, and another sub-task to fix what breaks.

None of this adds code. A user starts a run by naming the technique in the task text, and the model carries out the steps with the tools of Section 2.3: Bash for the profile and the evaluation, the browser for the search, `run_parallel` for the break and fix sub-tasks, `Write` for the two logs. The state of the search is in the two files, so a sub-session the Relentless Agent starts from a summary can read what was tried and what failed; the rule that a failed idea is never retried is meant to keep a day-long run from circling. The testing rules of Appendix C apply to every implement step, and the held-out check guards against tuning to the one workload the loop optimized on.

The seven-step text is the current one; the Bespoke OLAP task of Section 4.2 ran under an earlier wording of the same steps (Appendix F). The HydraKV tasks ran under an earlier paragraph (Appendix F) that asks the agent to search, implement, measure, and note tried ideas and metrics in a file; it names no files and has no numbered steps, pairwise judging, or adversarial-testing block. HydraKV’s adversarial steps came from its task prompt, and its state lived in `PROGRESS.md` and, from the first hardening task on, `DISCOVERY_LOG.md`.

3 Evidence

The claims of Section 2 and Appendix C are design claims. This section reports what can be measured: what six months of the agent’s own task database show (Section 3.1), what a controlled ablation of the system prompt shows on small tasks (Section 3.2), and what adding a read-only reviewer from a second vendor does (Section 3.3). Section 3.4 lists what these measurements

cannot show. All numbers are recomputed by the scripts in `papers/kissorcar/evidence/` and `papers/kissorcar/ablation/`; each macro in the source names the JSON key it is read from.

3.1 Six Months of Self-Hosted Use

The Chat Sorcar Agent persists every task, its settings, and its result to `sorcar.db`, and, when the task runs in a chat webview of the VS Code extension or the web app, every tool call and tool result (Section 2.4). The database on the development machine, snapshotted on 18 September 2026, holds 23,438 tasks: 5,574 top-level tasks in 2,324 chats and 17,864 sub-agent tasks (parallel sub-agents, reviewers, summarizers), spanning 165 released versions from April to September 2026, \$29,593 of model spend, 19.9 billion tokens, and 237,000 agent steps. 5,337 of the top-level tasks (96%) ran in this repository, at a cost of \$29,234; the rest were papers, benchmarks, and the two case studies. The distribution is long-tailed: the median top-level task cost \$0.50 and took 2 steps, the ninetieth percentile \$10.24 and 96 steps, and 512 tasks ran past 100 steps (the longest 8,056; the most expensive \$845). Two caveats apply throughout. The database records one user’s work on mostly one repository, so it measures how the framework behaves under its author’s prompts and says nothing about how it generalizes. And tool-call timestamps were not persisted before July 2026, and 1,914 sub-agent rows from July 2026 carry a process-kill notice in place of a result, so per-month series start in July and review outcomes are available for a subset (below).

Continuation. 773 tasks used the Relentless Agent’s continuation at least once: 528 once, 16 ten or more times, and one task 119 times. The median continued task cost \$3.39, seven times the median task. Continuation is what a long task needs and what a short task never sees; four of the 8 HydraKV tasks of Section 4 are in the tail; the Bespoke OLAP tasks finished within one context window.

Steering and isolation. 400 top-level tasks received 480 follow-up messages while running (Appendix B.2), and 298 tasks were stopped by the user. 48% of top-level tasks ran in a git worktree; 1,023 were auto-committed and squash-merged when they finished. The rest ran with isolation off, before the worktree layer existed, or, since September 2026, were classified as not editing files.

Read before edit. Of 28,832 `Edit` calls whose result was recorded, 173 (0.6%) targeted a file the agent had not read, written, or viewed through a shell command earlier in the task. 661 (2.3%) failed with “string not found”: the agent had read the file, then edited from a stale memory of it. In the versions the database covers, through 2026.9.18, the `Edit` tool did not enforce the rule; it only reported the miss (Section 2.3 describes the check added in 2026.9.19). Section 3.2 measures what removing the rule does.

Tests without mocks. The agent wrote 1,376 Python test files with the `Write` tool. 11 of them import `unittest.mock`, `pytest-mock`, or a patch decorator, and 10 `Edit` calls added such an import to a test file; none of the 11 were written after June 2026. A further 158 use `pytest`’s `monkeypatch` fixture without a mock library, most often to set an environment variable or a working directory, which the rule does not forbid.

Parallel test sharding. The agent called `run_parallel` 3,920 times; 990 of those calls split a test run across sub-agents, with a median of 4 shards and a ninetieth percentile of 12.

Model switching and review. `set_model` was called 4,152 times, 1,771 of them to `gpt-5.6-sol` and 1,649 back to `claude-fable-5`: the develop-then-review pattern of Section 2.3 executed inside one task. 707 top-level prompts named models from two vendors. 4,112 sub-agent tasks were asked to review, 2,573 of them read-only; the median review cost 13% of its parent task and the ninetieth percentile 37%. For the 1,675 read-only reviews whose result text was persisted, we asked the typed decisions model behind the agent’s `decide` tool (Section 2.3) two questions about each result: whether it reports at least one concrete defect, no defect, or could not complete; and whether it cites files, lines, tests, or command output. 71% report at least one concrete defect, 17% report none, and 12% did not complete; 80% cite evidence. Where the parent prompt named two vendors (611 reviews), 78% report a defect and 90% cite evidence, and 372 of the 477 defect-reporting reviews were followed by edits in the parent task, against 48 of the 113 that reported none. We hand-checked a random sample of twelve verdicts and agreed with ten; one disagreement was a fix report that the keyword filter had admitted as a review, the other an audit of repository branding whose findings were observations with no defect among them. A reviewer that reports a defect in seven of ten reviews is either finding real problems or inventing them; the controlled comparison of Section 3.3 separates the two.

Table 2: System-prompt ablation on 10 tasks, 2 repetitions each, `claude-fable-5`. *Hidden*: cells in which every hidden test passed. *Unread*: cells with at least one `Edit` of a file the agent had not read. *Mocks*: cells whose test files import a mock library. *Shards*: cells that split a `pytest` run across two or more `run_parallel` sub-agents. *Tests*: median number of test functions the agent added. *Steps* and *wall* are medians; *cost* is the mean per cell in USD, sub-agents included.

Arm	n	Hidden	Unread	Mocks	Shards	Tests	Steps	Wall (s)	Cost
<code>SYSTEM_LITE.md</code> (630 words)	20	20/20	8	0	0	0	6	51	0.38
<code>SYSTEM.md</code> (full prompt)	20	20/20	0	0	2	9	9	91	0.83
full minus read-before-modify	20	20/20	0	0	5	9	8	78	1.12
full minus no-mocks / end-to-end-only	20	20/20	1	0	7	10	12	96	1.42
full minus parallel test sharding	20	20/20	0	0	0	9	9	84	0.70
full plus cross-vendor read-only review	20	20/20	1	0	2	11	26	280	1.91

3.2 Ablating the System Prompt

Design. We wrote 10 self-contained Python tasks (Appendix E): each is a fixture module of 23 to 44 non-blank lines with three or four passing tests, a one-paragraph task that adds a feature or fixes a bug, a hidden test file of 7 to 9 tests that all fail on the fixture and all pass on a reference solution the agent never sees, and no dependency outside the standard library. Each cell copies the fixture into a fresh git repository, runs `SorcarAgent.run` on it with `claude-fable-5`, web tools and memory off, and a budget cap of \$8 (\$12 for the review arm), logs every tool call of the top-level agent and of its sub-agents through the framework’s `tool_call_hook`, and then runs the hidden tests and the fixture’s own tests against the result. Six arms differ only in the system prompt or in a clause appended to the task: the `SYSTEM.md` shipped at version 2026.9.18; the 630-word `SYSTEM_LITE.md` of version 2026.9.18 that the pre-run classifier uses for simple tasks, which has the identity and output contract and none of the engineering rules; `SYSTEM.md` with the read-before-modify sentence removed; with the no-mocks and end-to-end-only clauses removed; with the mandatory parallel-test-sharding bullet removed; and `SYSTEM.md` with the cross-vendor review promptlet of Section 2.3 appended to the task. Blocks are removed by exact string match, so a drift in `SYSTEM.md` fails the run, so an arm can never silently ablate nothing. Every cell ran 2 times, 120 cells in all; the records, sandboxes, and analysis script are released.

Result. Table 2 has one finding a reader of Appendix C might not expect and several they would. Every arm passed every hidden test in every cell: on tasks of this size, with this model, none of the prompt rules changes correctness, and neither does removing all of them. What the rules change is how the agent works. Under the short prompt the agent wrote tests in 2 of 20 cells and edited a file it had not read in 8; under the full prompt it added a median of 9 tests per cell and never edited an unread file, at $2.2\times$ the cost and $1.8\times$ the wall-clock time of the short prompt. We also ran `ruff` (rules E, F, W, default configuration) on the final source of every cell: 15 to 18 cells per arm have findings, but 242 of the 379 findings over all cells are the 21 that our own fixtures already carry (matched by file, rule, and line text), so the count does not separate the arms and the table omits it. Removing the read-before-modify sentence produced no unread `Edit` either, but two cells overwrote an unread module with `Write(reldate.py, kvstore.py)`, against none under the full prompt: the `Edit` tool needs an exact string from the file, so the model reads before editing whether or not it is told to, and any effect of the rule is confined to whole-file rewrites, where two cells against none does not separate the arms (since 19 September 2026, after the ablation ran, the tools themselves reject an unread file, Section 2.3); the other columns of that arm differ from the full prompt by amounts within the run-to-run spread. Removing the no-mocks clauses produced no mocks either; these tasks have nothing to mock. Removing the parallel-sharding bullet removed the sharding (0 cells against 2), and the bullet itself was followed in only 2 of 20 full-prompt cells, because the fixtures have three or four tests and the bullet applies to whole-suite runs. The review arm is the subject of the next subsection. In sum, the system prompt’s engineering rules are visible in the process and in the tests the agent leaves behind, and invisible in hidden-test correctness at this task size. The case studies of Section 4, where the tasks are thousands of lines and the defects that mattered were found by review, are the setting the rules were written for, and the one this ablation does not reach.

3.3 A Read-Only Reviewer from a Second Vendor

The review arm appends the first promptlet of Section 2.3 to the task: develop with `claude-fable-5`, then dispatch a read-only `gpt-5.6-sol` sub-agent through `run_parallel` to look for missed code, wrong wiring, and introduced bugs, under half the budget and with the instruction not to invent problems. We wanted a second vendor because a model favors its own output [Panickssery et al., 2024] and does not correct its own reasoning without external feedback [Huang et al., 2024]. The reviewer reported 21 distinct findings across the 20 cells (none in five cells: both of `http_retry`, both of `ini_parser_fix`, and one of `todo_sqlite`, where it returned “no issues found”), and the developer model fixed each before finishing; the hidden tests, which every arm passed, covered none of them. The reported defects are of the kind a second reader finds: `float("NaN")` accepted as a numeric CSV value; a crash on a self-referential symlink cycle in a directory walk; a bytes-returning `PathLike` that produced a malformed backup name; a timeout that killed the child process and left its descendants running, so the pipe stayed open; an `OverflowError` on an id past SQLite’s 64-bit range; an error message that used `repr` and so did not contain the input it quoted; an on-disk envelope that a legacy file could collide with. A finding is only as good as its reproduction, so we checked each one against the task text and wrote a probe for it: a script that exercises the reported case on a given artifact and exits 1 if the defect is present (Appendix E). By the task text, 8 findings are specification violations, 9 are defects the specification does not cover (a crash on a symlink cycle, an undecodable byte in a subprocess’s output, a clock that runs backwards), and 4 are not defects (two ask for behavior the reference solution also lacks, one is a style remark, one concerns `clock()` against `self.clock()` on the same object). Running the 17 real-defect probes against the 10 artifacts of the other five arms for the same task and against the reference solution: 15 reproduce on at least one artifact built without a reviewer, 12 on a full-prompt artifact, and 11 on our own reference solutions, which we wrote before the experiment and leave as they were so that the records stay reproducible. The remaining 2 existed only in the review cell’s own first draft, which the reviewer caught and the developer overwrote. Counting cells, the reviewer reported at least one real defect in 15 of 20. Four reviewer misses are also on record: one reviewer’s process-group fix was partial (a child that exits in time but leaves a grandchild holding the pipe was still reported as timed out), and three defects that one repetition’s reviewer caught, the other repetition’s reviewer did not. The review costs $2.3\times$ the full-prompt cell and $3.1\times$ its wall-clock time.

Two conclusions follow, and the observational data of Section 3.1 support both. A second model reading the diff finds real defects that the first model’s tests miss, at a cost the promptlet caps, and about one finding in five is not a defect, which is why the promptlet keeps the reviewer read-only and leaves the developer model to judge and fix; in the database, the 1,675 read-only reviews with a persisted result report a defect 71% of the time and the parent acts on most of them. And hidden tests written before the task are the wrong instrument for measuring that value: the defects above are boundary cases that a specification-driven test author does not think of either, which is why they survived both the developer model and our hidden tests and fell only to a reader looking for them.

3.4 Threats to Validity

The database is one developer’s use of one repository with a handful of frontier models; the mocks and unread-edit rates say how this agent behaved under these prompts, and a user who asks for mocks will get them. Sub-agent rows before August 2026 often lack a persisted result, so the review classification covers 1,675 of 2,573 read-only reviews, and the classifier is a model, checked by hand on twelve cases. The ablation tasks are small by construction (so that hidden tests can be complete and a cell costs about a dollar), which is why they show no correctness effect; a task that spans files, days, and context windows might, and we have not run one under controlled conditions. All ablation cells used one developer model and one reviewer model; the finding that a reviewer from a second vendor catches what the first misses is about `gpt-5.6-sol` reading `claude-fable-5`’s code, and a same-vendor reviewer was not measured here. The probes that verify reviewer findings were written by us after reading the findings, so they test whether a reported defect is real and present elsewhere; they do not test whether a reviewer would find every defect that is there.

4 Case Studies

The procedures of Section 2.6 were written for tasks that take days, and the two projects below are the runs of that kind for which we have a complete record, both released under `projects/`: HydraKV, a larger-than-memory key-value store, and the released TPC-H engine of the VLDB’26 *Bespoke OLAP* paper made $34.5\times$ faster than its single-threaded artifact. Each had a task prompt with a numeric goal, `claude-fable-5` developing, and `gpt-5.6-sol` reviewing read-only (capped at 20% of the task budget in the HydraKV tasks, and run as a separate review task for Bespoke OLAP). Table 9 (Appendix F) gives their process record mined from `sorcar.db`. HydraKV took 8 tasks: one wrote the starting engine, the adversarial discovery task (3.8 hours, \$81) ran the loop its task prompt describes under the earlier one-paragraph instruction, and six tasks followed (five of bug hunting, hardening, and audit response, one saving the code and its scripts), 22.6 agent hours in all; the Bespoke OLAP engine took one 1.3-hour optimization task under the seven-step loop without its web search, a separate read-only review task, and an audit task that wrote adversarial parameter tests. The authors wrote no code in either project; the human contribution was the task prompts, occasional steering messages, and forwarding the findings of external auditors. For each project we read the record for three things: whether the code that shipped was tested as the rules require, whether the agent followed the procedure its prompts stated, and what the second-vendor reviewer found that the benchmark did not. These are case studies of one run each on tasks we chose; they do not separate what the framework contributed from what the model did.

4.1 HydraKV: a Key-Value Store Built under an Adversarial Loop

Task. The benchmark description (`benchmark/TASK.md`) fixes everything except the engine: a YCSB-A [Cooper et al., 2010] trace of 250 million 64-bit keys with 100-byte values (50% reads, 50% blind upserts, Zipfian $\theta=0.95$), about 25 GB of data, 16 worker threads, a 30-second scored window from a cold cache, the median of three runs, an 8 GiB resident-memory cap the harness enforces, a validation pass that must find every loaded key, and a read-after-write integrity auditor; a run that exceeds the cap, hangs, or fails the auditor scores zero. The figure to beat was the 0.93 million operations per second (Mops/s) that the description states for Microsoft’s FASTER [Chandramouli et al., 2018]. No FASTER run log exists, FASTER was not run on our machine, and RocksDB, SplinterDB, and Kvell were not run either, so the comparison below is between our measured engine and a figure in the task text, on one workload under one memory cap. The specification forbids network access, reading outside the task folder, precomputing from the traces offline, regenerating or compressing values, and returning anything but the stored bytes; the ban on deriving constants from the key population came from the user’s steering messages. The machine was a GCP `n2-standard-64` (64 vCPUs, eight local NVMe SSDs in RAID0), reached over SSH from a KISS Sorcar task.

The adversarial task. The starting point was an engine that a KISS Sorcar task under `claude-fable-5` had written earlier the same day (\$12, 45 steps): the task followed the benchmark repository’s setup steps, wrote `hydra.cc` from first principles with no discovery loop and no adversarial workloads, and its one steering message said there was no need to keep improving the engine. It reward hacked the benchmark: it indexed keys in a flat four-byte-per-key array over fixed 128-byte log slots, which works only because the published trace uses keys below 2^{32} ; it validated all 250 million keys and reached 1.85 Mops/s. The adversarial-discovery task prompt (Appendix F) asked for adversarial AI discovery: generate workload variants the engine was not tuned for, break it, fix it, and finish on a fresh held-out variant with no tuning. The task ran 3.8 hours and \$81 across one continuation and received four steering messages: a warning against fitting the trace, a mid-task change of goal to 10 Mops/s, and twice “do not hard-code 250M dense keys”. The agent’s first move was a trace generator (`gen_variant.cc`) with sparse SplitMix64 keys and scrambled, clustered, drifting, and mixed hot sets. The first variant, sparse keys under the original skew, killed the inherited engine by out-of-memory before it could score; the replacement, a fingerprint hash index sized from the memory budget, scored 1.87 Mops/s on the original trace and 1.91 on the variant. The 10 Mops/s goal then drove a redesign led by the agent’s own diagnostics (a 72% cache hit rate; one contended I/O queue): log-append flushes, a doorkeeper admission filter, and 64 sharded I/O queues. Two intermediate builds regressed to 1.10 and 0.47 Mops/s; after profiling them and measuring the device with `fio`, the agent built eight raw `io_uring` rings driving an asynchronous read-miss state machine (5.27 Mops/s), then SLRU-style [Karedla et al., 1994] cache segmentation (5.50). Four further cache designs measured 0.78, 0.95, 1.98, and 4.17 Mops/s and were reverted, with their numbers recorded

in the task’s `PROGRESS.md`. Two read-only reviews (14 bugs and 10 risks, then 21 line-referenced findings, by the task record) were answered inside the task; the second found that a fingerprint alias, which is likely among 250 million keys, could abort the process or bury a live key behind a newer slot. The task’s final report gives 5.48 Mops/s on the original trace, 5.67 and 5.56 on the sparse and clustered variants, 3.97 on the drifting variant, and 3.87 on the held-out mix (Table 3), all with validation passing; these five numbers rest on the task record, since the harness logs stayed on the benchmark server. The 10 Mops/s goal was not met. The agent’s closing argument was that 50% reads at 10 Mops/s need a hit rate of about 86% ($1 - 0.718/5$; its summary rounded this up to 87%) under the measured device ceiling of 718k random 4 KiB reads per second, that every admission policy it tried saturated near 77%, and that getting past that rate would take cache memory outside the 8 GiB cap; at that hit rate the device bounds throughput near 6.2 Mops/s, and the standalone HydraKV paper in `papers/kvstorepaper/` places the engine at about 89% of that bound.

Hardening and audits. Six more tasks followed, each opened by the user with a request or an external audit. The first (4.8 hours, \$152, 561 steps, two continuations) asked the agent to find every redundancy, corner-case bug, race, and deadlock in its best engine, to reach full test coverage with end-to-end workload tests, and to keep the score at or above 5.48 Mops/s. The next (5.8 hours, the longest task of the project) answered a production-readiness audit. Its discovery log lists 5 kept idea groups (slot layout, crash recovery, fail-soft I/O, bounded oversized values, log compaction by hole punching) and five rejected with reasons; the tests were written before the fixes; and a second adversarial round found a deleted-key resurrection bug after the full test matrix had passed. A second external audit (`AUDIT2.md`) confirmed the score on the reference hardware and reproduced 4 bugs that the scored benchmark cannot reach: a deadlock between concurrent upserts and deletes, an out-of-memory on a full disk, a recovery that dropped 480K of 976K keys and reported success, and deletes 160 times slower than upserts (244 μ s against 1.5 μ s). A same-day A/B on the reference node gave 5.50 Mops/s for the fixed engine and 5.50 for the pre-fix engine. A workload task then ran the engine over 7 mixes (Table 8, Appendix F), two of them (0:100 and 5:95 read:write) quoted from the user feedback in its task prompt, and its one steering message pointed at FASTER’s own test suite; the task ran 436 steps and cost \$87.92, and it fixed 8 bugs. 4 came from the workloads and the test matrix, among them clean-shutdown data loss present on every workload, oversized values with no disk path at all, and a stale tombstone that could delete a key it did not belong to; the other 4 were found by the reviewer, and no workload had triggered them. The last audit found that the fix for slow deletes had introduced crash-resurrection of deleted keys: a delete returned once the key was queued for a background reaper, so a kill before the reaper ran let recovery bring the key back (5,000 of 5,000 deletes in the minimal reproduction). The repair added a delete-intent log ahead of the acknowledgment; two review rounds on it returned 14 findings and then nine more on the round-one fixes, among them a linearization race (an upsert publishing between the delete’s registry mark and its cache sweep), which moved the delete’s linearization under the key’s set lock. The final engine measures Delete at 1.0 μ s, the cost of an upsert. Two agent failure modes stood out, and both were provenance failures: in the bug-hunting task the agent layered correctness fixes onto a rejected, slower branch (scored 3.6 Mops/s until a one-sentence steering message, “did you start with most performant variant of the engine?”, prompted a diff against archived revisions), and it once retried an idea its log had already marked as failed. The “never retried” clause of the current discovery loop is a direct consequence.

What the agent built. The engine composes known mechanisms; none of them is individually new. Values live in fixed 128-byte slots of a single append-only `O_DIRECT` log, each slot sealed with a 56-bit log sequence number (LSN) and a CRC-32C computed with the SSE4.2 `crc32` instruction. Writer sessions reserve disjoint 8,192-slot extents, so slot position is not a global write order; the LSN is the authoritative version order, and recovery, multi-version reads, and compaction’s position reuse all depend on it. A cache-line-bucketed fingerprint hash index (32-bit fingerprint plus 32-bit slot position per 8-byte word, lock-free CAS insertion, a quarter of the memory budget) maps keys to log positions; because fingerprints collide among 250 million keys, no read trusts the index, and every candidate is verified against the full key stored on disk. The remaining budget is an 8-way set-associative write-back cache of 120-byte entries, about 47 million values, with two admission mechanisms against the Zipf tail: a two-generation doorkeeper bitmap in the spirit of TinyLFU [Einziger et al., 2017] (a read miss is admitted only if its key was seen recently) and SLRU-style probation and protected ways with one-bit second-chance recency. Blind upserts of uncached keys are absorbed as dirty entries and written back lazily by four cleaner threads under a pin-ownership protocol that lets a concurrent upsert or delete invalidate a staged flush. Read misses go to eight dedicated I/O

Table 3: HydraKV throughput on the reference machine (16 workers, 8 GiB budget, cold cache, 30 s runs). Scored rows are medians of three runs. The three operation-mix rows at the bottom are single runs under the scored protocol: the write-only row is transcribed from the node’s run report of engine 5050e98f (refnode_workloads_jul21/README.md); the last two are from the workload task’s record (WORKLOAD_HARDENING.md), which names no engine revision, and were not re-run. The adversarial variants and the holdout were scored with the engine revision that closed the discovery loop (v4c), and their numbers come from that task’s final report; only the original workload was re-scored with the later, hardened revisions. The FASTER row is the figure stated in the task’s benchmark description; FASTER was not run on this machine.

Engine	Workload	Mops/s
FASTER (as stated in the task)	YCSB-A, Zipf $\theta=0.95$ (scored)	0.93
starting engine (dense index, written by one Sorcar task)	scored	1.85
HydraKV, v4c	scored	5.48
HydraKV, v4c	V1: sparse SplitMix64 keys	5.67
HydraKV, v4c	V2: clustered hot set	5.56
HydraKV, v4c	V3: drifting hot set	3.97
HydraKV, v4c	holdout: 60/25/15 mix, $\theta=0.92$	3.87
HydraKV, c7745c61	scored (July 21 reference-node re-run)	5.50
HydraKV, 5050e98f	scored	5.51
HydraKV, 5050e98f	0:100 write-only	5.12
HydraKV, workload task	95:5 (YCSB-B)	2.93
HydraKV, workload task	100:0 (YCSB-C)	2.78

threads, each owning a hand-rolled `io_uring` ring (the machine had no liburing), with a synchronous `pread` pool as fallback. Around this hot path sit the subsystems the audits asked for: scan-based crash recovery that rebuilds the index newest-LSN-wins, fail-soft I/O error handling with sticky error counters, background log compaction with hole punching and position reuse, multi-slot records for oversized values, and a crash-durable delete path. The engine contains no constant derived from the trace’s key count, key density, or skew.

Result, and what it does not show. The released engine is 4,324 non-blank lines of dependency-free C++17 in one translation unit, with 28 self-asserting end-to-end tests (2,112 lines) that use real fault injection (an 8 MB file-size limit so that log writes fail with EFBIG, as on a full disk; a checkpointing child killed with SIGKILL; external truncation of the log). On the reference node, at commit c7745c61 before the workload fixes above, it sustained 5.50 Mops/s ($5.9\times$ the stated FASTER figure) with zero auditor failures and under the memory cap, and its Linux test matrix passed under ASan, UBSan, and TSan with 92.5% line coverage; after them (commit 5050e98f) it measured 5.51 Mops/s and 89.3% coverage in runs whose logs stayed on the node. $5.9\times$ compares the scored YCSB-A mix against a figure in the task text. On other mixes of Table 8, run for correctness without tuning, the engine is far slower: 0.30 Mops/s on `RMW_100`, all read-modify-writes, the pattern FASTER’s in-place updates target, and 0.08 on `TIMESERIES_HD`, which inserts at the tail of a key range and deletes at its head. The drifting variant and the held-out mix ran at 3.97 and 3.87 Mops/s, more than a quarter below the scored trace, so the cache policy is sensitive to how the hot set moves. “Production-hardened” is the agent’s phrase, and the record argues against it: three external audits each found bugs the previous round had introduced or missed. The record does support the shape of the loop: the agent’s variants broke the inherited engine before any human did, four designs were measured and discarded, and the developer answered each of the 11 read-only reviews in the same task. Most findings were fixed in code. Four in the discovery task that the harness could not exercise (Delete, oversized values, restart recovery, log garbage collection) went into its progress file, and of the 18 findings of the production-readiness review, 12 were fixed and 6 were documented as residual risks in `PROD_READINESS.md`.

4.2 Bespoke OLAP: a TPC-H Engine Made 34.5× Faster

Task. The VLDB’26 paper *Bespoke OLAP: Synthesizing Workload-Specific One-size-fits-one Database Engines* [Wehrstein et al., 2026] synthesizes a C++ engine for the 22 TPC-H query templates: the templates are fixed, only their parameters vary, and the engine is allowed to exploit that at build time. Its released artifact runs single-threaded and, on our 32-core Linux machine, needs

Table 4: Bespoke OLAP TPC-H engine before and after optimization: sum of per-query median execution times over five repetitions on the 32-core machine; all configurations validate against DuckDB with an empty error set. Seed 999 was held out during development. Ingest is not timed by the benchmark.

Configuration	Released artifact	Optimized	Speedup
SF10, seed 42	3,132.2 ms	90.8 ms	34.5×
SF10, seed 7	—	92.5 ms	
SF10, seed 123	—	82.1 ms	
SF10, seed 999 (held out)	—	85.1 ms	
SF1, seed 42	276.8 ms	14.2 ms	19.5×
SF10 ingest (not scored)	52.5 s	63.0 s	

3,132.2 ms for the 22 queries at scale factor 10 (SF10; sum of per-query medians over five repetitions, seed 42) after a 52.5 s ingest. The task given to KISS Sorcar was a repository-optimization prompt of the kind in Appendix A.4, with a numeric goal (at most 783 ms, a $4\times$ speedup, with an empty validation-error set on six scale-factor and seed configurations) and six hard constraints: modify only the engine/ directory; keep the stdin request protocol, the CSV output, and the timing window (start before parameter parsing, stop after the executor returns, before the CSV write) byte-for-byte identical; no result memoization across requests, because the harness repeats each request; any precomputation at ingest must be independent of the placeholder values of any request and may exploit only the schema, the data distributions, and the fixed template structure, which is the paper’s own “DBMS contract”; every query must remain correct for arbitrary valid placeholders; and aggregates keep the baseline’s numeric types. Results are validated against DuckDB reference outputs sorted by all columns with an absolute and relative tolerance of 10^{-2} , the policy of the paper’s own validator. The prompt was directive: it named the AffinityGuard that pinned the process to one CPU, prescribed OpenMP as the first round and precomputed join artifacts as the second, and listed the heaviest queries. The development task ran 1.3 hours, 162 steps, and \$43 in one context window, with no continuation and no steering; the loop’s web-research step was not asked for and did not run.

What the agent did. The first round removed the pin, added `-fopenmp -march=native` through the build’s `extra_flags.txt` hook, and parallelized the scan and join kernels with thread-local partial results merged at the end. The second round moved parameter-independent work to ingest, where it is not timed: date-indexed prefix tables and cubes, with extra small-domain dimensions where a template has them (region for Q5; discount and quantity buckets for Q6; the two nations for Q7; nation, region, and part type for Q8; the two ship modes for Q12), for the queries whose free parameters are a date window and such dimensions (Q1, Q4 to Q8, Q12, Q14, Q15), which reduce to a difference of two prefix entries; partkey join indexes in compressed-sparse-row form for Q9, Q17, and Q20; per-order aggregates for Q10 and Q18; rows pre-filtered by the templates’ fixed predicates for Q13, Q16, Q19, Q21, and Q22; per-order segment codes for Q3 and per-nation value lists for Q11; and Q2 left on its original path. Every fast path checks its assumptions and falls back to the baseline algorithm when a request’s parameters fall outside what the precomputed artifact covers. Profiles drove the last changes: a serial 60-million-row scan hidden in Q19 and serial radix sorts in Q10. The task ended at 86.8 ms and $36.1\times$; after the review fixes below the figure is 90.8 ms and $34.5\times$.

Result. Tables 4 and 5 give the outcome. 10 of the 22 queries now take under 0.1 ms, which is what a cube lookup costs; the four heaviest baseline queries (Q9, Q12, Q18, Q21, 56% of the baseline total) take 11.0 ms together. Q10, a top-20 over customers with a date-range parameter, is now the largest remaining contributor at 43% of the total. Only Q2 (1 of 22) is slower than baseline, by 0.3 ms on a 4 ms query whose parameters do not admit a small prefix structure. The result held at 82.1 to 92.5 ms on the two other development seeds and at 85.1 ms on the held-out seed, which tracks seed 42 query by query.

What the number means. Removing the pin let the engine use 32 cores, but we did not measure the two rounds separately, so the split between parallelism and precomputation is unknown. The project’s README quotes 284 ms for the authors’ own multithreaded follow-up on the same machine, about $3.1\times$ behind the engine, but no run log of it was kept, so the $34.5\times$ we report is against the single-threaded artifact only. The prefix cubes are what the paper’s contract invites, since the templates are fixed by construction, but they move work to ingest (52.5 to 63.0 s, 20%), so the extra ingest is repaid after 4 runs of the query suite, and they answer only these 22 templates; a query outside the

Table 5: TPC-H at scale factor 10, per-query median execution time in ms over five repetitions on the 32-core machine: the released single-threaded artifact (seed 42), the agent’s final engine on seed 42, and the same engine on seed 999, which was not among the three seeds the task named. Speedup is baseline over final on seed 42; the text lists the mechanism behind each query. Per-query rows may not sum to the totals because of rounding.

Query	Baseline	Final	Seed 999	Speedup
Q1	161.1	0.007	0.007	23,015
Q2	4.2	4.466	4.380	0.9
Q3	137.5	8.624	8.089	15.9
Q4	13.0	0.020	0.019	650
Q5	33.7	0.009	0.009	3,741
Q6	33.0	0.010	0.009	3,303
Q7	74.4	0.006	0.007	12,408
Q8	82.4	0.008	0.010	10,299
Q9	622.2	3.719	2.606	167
Q10	104.5	39.054	33.913	2.7
Q11	16.2	0.428	0.331	37.9

Query	Baseline	Final	Seed 999	Speedup
Q12	377.2	0.008	0.008	47,156
Q13	219.5	9.560	9.129	23.0
Q14	43.2	0.007	0.008	6,167
Q15	103.0	0.810	0.782	127
Q16	34.5	10.050	11.712	3.4
Q17	64.1	1.365	1.421	47.0
Q18	372.3	2.376	2.190	157
Q19	216.8	0.050	0.045	4,336
Q20	41.7	5.296	5.431	7.9
Q21	367.9	4.944	5.025	74.4
Q22	9.7	0.014	0.012	691
Total	3,132.2	90.8	85.1	34.5

templates has no fast path at all, and nothing here is a general OLAP result. The results are from one machine with one compiler, a same-box, same-harness comparison. The task prompt supplied a researched plan and the agent executed it, which is the loop’s implement-and-measure step without its search steps; the agent’s own contributions are the choice of artifacts per query, the fallbacks, the profile-driven fixes, and the 265-case audit below.

How the constraints shaped the work. Each hard constraint in the prompt closes a shortcut that an optimizing agent would otherwise take. Keeping the timing window fixed forbids moving parameter parsing or per-request warm-up out of the measured region; the reviewer confirmed, for every one of the 22 dispatch branches, that `parse_qN` and `run_qN` sit between the two `now()` calls and the CSV writer after them. Forbidding memoization matters because the harness repeats identical requests five times to take medians, so a request-keyed cache would turn the benchmark into a measurement of hash lookups; the reviewer’s static-state scan of all 22 executors found only immutable month-length tables and tracing-only `TraceData` objects. Requiring parameter-independent precomputation is the line between exploiting the paper’s contract and exploiting the benchmark seeds; the only literals in builder code are values fixed in the templates themselves (PROMO for Q14, the 1995–1996 window for Q7 and Q8, the return flag R for Q10). The constraint that aggregates keep the baseline’s numeric types is the one the agent violated, in the Q1 accumulator, and the one the reviewer caught.

Review and audit. A separate task on `gpt-5.6-sol`, told to modify nothing and given 22 steps (\$2.55), worked through six checks in priority order: timing-window integrity, absence of cross-request memoization, parameter independence of the precomputation, races and numeric boundaries, dispatch completeness, and output fidelity. Five passed with file-and-line evidence; the fourth failed on two defects. In Q20, a parallel loop over matching parts marked qualifying suppliers in a shared byte vector without synchronization, a data race whenever a supplier qualifies through more than one part, which the SF10 references (1,738 to 1,830 result rows) make routine; both racing stores write the value 1, so results were correct, but the access is undefined behavior, and the fix is an atomic store. In Q1, the new prefix cube accumulated `sum_charge` in `int64_t` where the baseline uses `__int128`; the reviewer showed that this leaves about $830\times$ headroom over SF10 and placed the overflow near scale factor 8,300, a scale the task never runs but the engine claims to support. Neither defect is visible to the DuckDB validation at SF10, and both were fixed before the final measurements. A third task then audited the fast paths for arbitrary parameters. It built a `-DBESPOKE_AUDIT` variant that emits a marker on every guard and fallback, and wrote 265 request cases (boundary dates and leap-day month arithmetic, out-of-dictionary strings, numeric extremes, duplicate IN-list values, empty results, and for every reachable guard a case in which the fallback engages), each compared against the pristine artifact as oracle. All 265 pass; the one finding is a contract note, that Q22 truncates phone codes longer than two characters as the artifact does. The audit’s own read-only reviewer found no engine defect but three holes in the test harness, which the developer fixed before the 265-case figure was taken. The audit task took 0.97 hours, \$45, and 291 steps.

What the two studies have in common. In both projects the speedup came from the agent’s reading of the problem’s contract: HydraKV composes log-structured storage, fingerprint indexing, and

cache-admission ideas from the caching literature; the Bespoke engine exploits the fixed-template contract the original paper states but does not fully use. In both, the developer model’s own tests, sanitizers, and validation harness missed the defects that mattered (a delete-durability regression, an aliasing fingerprint, an OpenMP race, a narrowed accumulator); those were found by a reviewer model from a different vendor or by an external auditor attacking the built artifact with its own oracle. And in both, the artifacts the task left behind (a discovery log, per-run logs, audit reports, and self-asserting tests) are what made the later audits, and this section, checkable.

5 Terminal-Bench 2.0 against Three Harnesses

The two case studies have no controlled baseline agent on the same task. The HarnessTax study of Pan et al. [2026] gives us one comparison with other harnesses. It holds the model fixed and varies the harness: seven models (the rows of Table 6) each run under Claude Code [Anthropic, 2025b], Codex CLI [OpenAI, 2025a], and Pi [Earendil Works, 2026] on the same 30 tasks sampled from Terminal-Bench 2.0 [Merrill et al., 2026] and 30 from SWE-bench Lite, three attempts per task, scored by each benchmark’s official verifier. Its conclusion is that the harness moves cost more than success, and that Pi, which gives the model four tools and little else, sits on the cost-success frontier. The study publishes its per-model numbers, task list, and protocol, so we ran our harness on the same 30 tasks under the same scoring; we chose Terminal-Bench 2.0 over SWE-bench Lite because its tasks exercise what our rules are about: long shell sessions, background processes, and container state the verifier inspects.

The benchmark agent is KISS Sorcar with the general-purpose instructions removed from its system prompt and the coding and software-engineering instructions retained: 727 words in place of the 3,851-word `SYSTEM.md`. That prompt leaves out the two procedures of Section 2.6, so the benchmark tests the loop, the tools, and the coding rules only. A Harbor [Harbor Framework Team, 2026] agent (Harbor 0.20.0) hands each trial’s container to the framework, whose tools run inside it: the built-in toolset of Section 2.3 with the browser, memory, and worktree off. The prompt (archived verbatim with the records in `papers/kissorcar/evidence/tb2_prompt.txt`) names the tools and states six rules of the kind Appendix C states, among them: list what “done” means for each item of the task, never modify the task’s input files, and before finishing check every requirement from a fresh shell. Two harness mechanisms back the rules in code. After each shell command the harness appends a note to the result when processes the command started are still running or when a pre-existing file under the working directory changed, and a command that would kill every process in the container or delete the workspace is refused with a message. Unlike the three harnesses in the study, we set no turn cap and no wall-clock limit: an attempt ends when the agent calls `finish` or has spent \$50. We ran 630 attempts (7 models, 30 tasks, three each) on September 25, 2026, and all of them count. Statistics follow the study: a task’s attempts are averaged, then the 30 tasks; pooled figures average the seven per-model results, and intervals are percentile bootstraps over 10,000 resamples of the tasks with the models weighted equally.

Table 6 and Figure 4 give the result. Pooled over the seven models, KISS Sorcar solves 75.6% of attempts ([63.8, 85.9] over tasks) against 70.0% for Pi, 65.7% for Codex CLI, and 65.1% for Claude Code, and its point estimate is the best of the four harnesses on 7 of the 7 models. With 30 tasks a per-model interval spans 27 points on average, so no single row settles its gap; the pooled gap over Pi is 5.6 points and over Claude Code 10.5, and the pooled interval covers all three published means. Counting solved attempts past the study’s 100-turn cap as failures, the pooled rate is 74.1%. These statements are about success. The two cost columns rest on different price lists (below), so we draw no cost ratio from them; on one price list and protocol, the paired Pi run below finds a cost difference of +0.05 dollars per attempt ([−0.71, +0.70]).

The trajectories show where the success and the cost come from. The mean attempt took 27.9 turns (longest 276) and 17.2 minutes; 28 attempts went past the study’s 100-turn cap and 9 of those were solved, and 3 cost more than \$15 (the most \$33.64), 2 of them solved. 250 of the 630 attempts received a harness note about a still-running process or a changed input file (901 in all), and the harness refused 15 tool calls in 15 attempts: 14 were `run_agent` calls, which have no channel agent to reach in the container, and 1 a shell command the guard misread. The cost comes from the number of turns, the prompt and tool schemas sent with every call, and the notes. The study ran each harness at high effort; we sent the providers’ default request parameters. Its table (September 22, 2026) is

Model	KISS Sorcar		Pi		Codex CLI		Claude Code	
	solved	\$/att.	solved	\$/att.	solved	\$/att.	solved	\$/att.
Claude Fable 5	80.0	1.50	71.1	1.08	72.2	0.98	75.6	1.55
Claude Opus 4.8	74.4	1.21	72.2	0.76	72.2	0.85	68.9	0.90
Claude Sonnet 4.6	76.7	2.23	65.6	0.61	63.3	0.55	62.2	0.67
Claude Haiku 4.5	51.1	0.39	47.8	0.25	31.1	0.21	41.1	0.26
GPT-5.6 Sol	85.6	0.69	83.3	0.42	78.9	0.76	71.1	1.35
GPT-5.6 Luna	78.9	0.05	76.7	0.04	72.2	0.06	70.0	0.10
Kimi K3	82.2	1.14	73.3	0.38	70.0	0.45	66.7	0.52
Mean of 7 models	75.6	1.03	70.0	0.51	65.7	0.55	65.1	0.77

Table 6: Terminal-Bench 2.0, the study’s 30-task sample, three attempts per task: percentage of attempts solved (mean of task means) and cost per attempt in USD, KISS Sorcar (630 counted attempts, no turn cap, \$50 budget) next to the three harnesses of Pan et al. [2026] as published (100-turn cap, high effort). Bold marks the best point estimate per row; intervals are in the text.

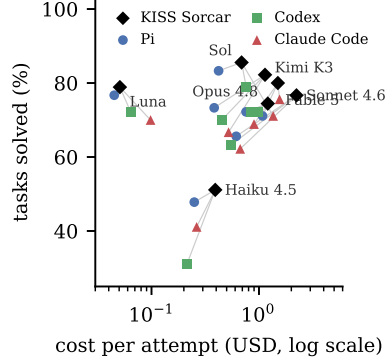


Figure 4: Table 6 as a cost-success plot. One diamond per model for KISS Sorcar; the gray lines join it to the same model under Pi, Codex CLI, and Claude Code.

priced at its September 1 list and ours at the framework’s price table, with two providers for Kimi K3. The seven rows share the 30 tasks, so they are seven views of one sample.

Pi on the same model, paired. To remove the date, price list, and cap from one row of Table 6, we ran Harbor’s Pi agent (Pi 0.87.1) on Claude Fable 5 on September 26, 2026: the same 30 tasks, three attempts each, at the study’s high-effort setting, under our protocol (no turn cap, no wall-clock limit; Pi has no budget cap, so an attempt ends when the model stops), its token counts priced by the framework’s table. All 90 attempts count, 1 of them stopped by us after two hours, while Pi waited on a training command it had started, and counted as failed with its spend. Pi solved 62 attempts, 68.9% ([52.2, 84.4]) at \$1.45 and 14.5 turns per attempt; KISS Sorcar on the same model solved 80.0% at \$1.50 and 15.3 turns. Both runs attempted every task, so the test is paired: the success gap is +11.1 points ([+3.3, +20.0], tasks resampled once for both harnesses), and the cost difference +0.05 dollars per attempt ([−0.71, +0.70], an interval that includes zero). KISS Sorcar has the higher task mean on 7 tasks, Pi on 0, and 23 tie, 20 of them solved by both on every attempt (exact sign test, $p = 0.016$). Pi’s 3 attempts on `feal-linear-cryptanalysis` all ended in an API error (two in a Pi client bug on a mid-output model fallback, one in a content filter), no answer submitted; without that task the gap is +8.0 points (6 tasks to 0, $p = 0.031$). Codex CLI and Claude Code were not rerun.

The rest of Terminal-Bench 2.0. We also ran the same agent and prompt on Claude Fable 5 on the 59 tasks the study did not sample, three attempts each, under the same protocol. One attempt, on `extract-moves-from-video`, was stopped before it finished and left no result; as with Pi’s stopped attempt, we count it as failed. The agent then solves 79.1% of attempts on the unsampled tasks and 79.4% over all 89 tasks. The intervals below use the 176 counted attempts (140 solved; 4 verifier timeouts count as failed), on which the rates are 79.7% ([70.1, 88.1]) and 79.8% ([71.9, 86.9]), against 80.0% on the study’s 30 tasks, a gap of +0.3 points ([−15.8, +15.4], each set resampled on its own). The unsampled tasks cost \$1.60 per attempt against \$1.50; their attempts are longer (17.2 against 15.3 turns) but cheaper per turn (\$0.089 against \$0.098, total cost over total turns: \$1.54 per attempt over 17.2 turns), and the rest of the gap is task weighting, the 2 counted attempts of `extract-moves-from-video` (\$13.27 each) carrying a full task’s weight.

6 Related Work

Code-specialized language models. Beyond the general-purpose LLMs that our system can use as backends, a rich line of work has produced models specifically trained for code. Code Llama [Rozière et al., 2023] fine-tunes Llama 2 for code generation and infilling. StarCoder [Li et al., 2023] trains on permissively licensed code from GitHub with a fill-in-the-middle objective. DeepSeek-Coder [Guo et al., 2024] trains on a 2-trillion-token corpus of code and natural language with a repository-level context window. Claude Opus 4.6 [Anthropic, 2026a] and its successor Opus 4.7 [Anthropic, 2026b]

advance long-horizon coding and agentic task execution; OpenAI’s GPT 5.5 [OpenAI, 2026b] similarly targets agentic software engineering with strong code generation and tool-use capabilities. Cursor released Composer 2 [Cursor Research, 2026], a custom fine-tuned coding model trained with large-scale reinforcement learning. Kimi K2.5 [Kimi Team, 2026] is an open-source multimodal agentic model that jointly optimizes text and vision and introduces Agent Swarm for parallel task decomposition. GLM-5.1 [Z.ai, 2026] is a 754-billion-parameter mixture-of-experts model from Z.ai that sustains autonomous coding over multi-hour sessions, achieving state-of-the-art on SWE-Bench Pro. Any of these models can serve as the backend without changes to the framework.

Code generation agents. SWE-Agent [Yang et al., 2024b] and OpenHands [Wang et al., 2025b] provide LLM-based agents for resolving software engineering tasks such as GitHub issues. SWE-Agent runs one session; OpenHands keeps one session and condenses it in place with an LLMSummarizingCondenser that replaces the middle of the conversation with a summary when a token threshold or a context error is hit [OpenHands, 2026]. Agentless [Xia et al., 2024] takes the opposite approach, showing that a fixed three-phase pipeline (localization, repair, patch validation) without autonomous agent loops can achieve competitive results on SWE-bench [Jimenez et al., 2024]. Devin [Wu, 2024] is an industrial product with shell, browser, and editor access in a sandbox. Claude Code [Anthropic, 2025b] is Anthropic’s terminal agent with shell and file access. OpenAI’s Codex [OpenAI, 2025a] runs coding tasks in cloud sandboxes with full repository context. Aider [Gauthier, 2023] provides a terminal-based pair programming interface with tight git integration, automatically committing each change. Our Relentless Agent restarts a fresh session from a summary instead of condensing in place, and our worktree isolation keeps every task’s edits off the user’s working tree, which per-change commits on the user’s branch do not.

Loops, continuation, and isolation in practice. Three pieces of practitioner work predate ours on the mechanisms of Section 2. Huntley [2025]’s Ralph loop (July 2025) restarts a coding agent from a prompt and a plan file until the plan is done, in one line of shell; the Relentless Agent is that loop with the plan file replaced by a summary the agent writes when it decides to hand off, and with the loop inside the framework so that budget, steps, and the chat history carry across. Karpathy [2026]’s autoresearch (March 2026) drives an experiment loop from a program.md and a results file, keeping or discarding each change by its measured effect; the single-prompt drivers of Appendix A are the same loop stated as a system-prompt block with a ledger of tried ideas, a held-out check at the end, and an adversarial sub-task. Claude Code’s `--worktree` flag [Anthropic, 2026f] and Cursor 2.0’s parallel agents [Cursor Team, 2025] give a session its own git worktree on request; the Worktree Sorcar Agent makes the worktree the default, keeps a spare ready, and reclaims worktrees left by crashed processes. The 32 messaging channels, pairing handshake, and gateway tick of Appendix B.6 follow OpenClaw’s catalog [OpenClaw AI, 2025]; the agents were written here, without OpenClaw code.

Long-running agents and discovery loops. mini-SWE-agent [SWE-agent team, 2026] is about 100 lines with a bash-only action space, a smaller loop than ours with no continuation or isolation. The discovery loop of Section 2.6 has older and larger relatives: FunSearch [Romera-Paredes et al., 2024] evolves a population of programs against a fixed evaluator; AlphaEvolve [Novikov et al., 2025] extends it to whole code files with an ensemble of models and evaluators; the AI Scientist [Lu et al., 2024] runs idea, experiment, paper, and review as one pipeline; and AIDE [Jiang et al., 2025] searches a tree of candidate solutions, refining the promising ones. There the loop is code that calls the model; in ours it is seven numbered steps in a prompt, run by the model with a shell and a browser, with the state of the search in two Markdown files. The adversarial sub-task descends from property-based testing [Claessen and Hughes, 2000]: the model writes the adversarial tests or workloads, and a second sub-task repairs what they break. Anthropic’s harness for long-running agents [Young, 2025] has each new context window read a feature list and a progress file first, the same state-in-files idea, and Carlini [2026] ran 16 Claude Code agents in a Ralph-style loop for about 2,000 sessions to build a C compiler that builds Linux 6.9, a long-running case study held on course by tests. StateM [Qin et al., 2026] is the closest system in aim: a runtime that keeps durable state, phase-local context, and versioned runbooks around a fixed model, and it reports same-model gains of eight to nine points on Terminal-Bench 2.1 (GPT-5.6 Luna from 76.7 to 85.4%) on all 89 tasks. Its state and procedures live in a runtime with checked transitions, ours in a prompt.

Measuring the harness. Several recent studies measure the harness or its prompt with the model held fixed. Ben Sghaier et al. [2026] find, across 35 releases of one CLI, that the harness alone moves SWE-bench Verified quality; Fan et al. [2026] ablate planning, action space, and context

management; Lin et al. [2026] localize an evolved harness’s gain to tools and memory, with none of it in the prompt; Zhang et al. [2026] find that the beneficial rules in a rule file are negative constraints. Sambharia [2026] coined the term harness tax for the per-request payload of tool definitions and instructions, and Pan et al. [2026] measured it across three harnesses and seven models, the study our Terminal-Bench 2.0 run reuses. Agarwal and Jain [2026] apply natural-language runtime policies and action denials at the harness level on Terminal-Bench 2.1; our harness notes and refused commands are policies of that kind, written by hand. Park et al. [2026] optimize a harness offline from failure traces on Terminal-Bench 2.0; ours is fixed for the run. Wang et al. [2026] find that a harness searched and scored on the same Terminal-Bench 2.1 tasks generalizes poorly to held-out ones and does not consistently beat test-time scaling at a matched budget; our run on the unsampled tasks bears on the first concern only. On review, Kim [2026] report from 375 artifacts of a Claude-and-Codex protocol that heterogeneous reviewer pairs record a defect more often than homogeneous ones (69.8% against 53.1%); our reviewer experiment of Section 3.3 measures a second-vendor reviewer against none, and does not compare it with a same-vendor one.

ReAct and tool use. The ReAct framework [Yao et al., 2023b] interleaves reasoning and action. Toolformer [Schick et al., 2023] teaches models to use tools via self-supervised learning. We use the native function calling of modern LLM APIs, which reduces prompt overhead.

Reasoning and planning. Chain-of-thought prompting [Wei et al., 2022] demonstrated that eliciting step-by-step reasoning improves LLM accuracy on reasoning tasks. Tree of Thoughts [Yao et al., 2023a] generalizes this to deliberate search over multiple reasoning paths. Reflexion [Shinn et al., 2023] introduces verbal reinforcement learning, where an agent reflects on failed attempts and produces self-critiques that improve subsequent trials.

Multi-agent software development. ChatDev [Qian et al., 2024] models the software development process as a conversation between role-playing agents (CEO, CTO, programmer, tester) organized in a waterfall-like pipeline. MetaGPT [Hong et al., 2024] takes a meta-programming approach, encoding standard operating procedures as structured outputs that coordinate specialized agents. AutoGen [Wu et al., 2023] provides a general-purpose framework for multi-agent conversation, enabling flexible topologies beyond fixed pipelines. We do not simulate an organization of specialists; we use a single agent with broad access to tools and optional parallel sub-agents for embarrassingly parallel sub-tasks.

Multi-turn autonomous agents. AutoGPT [Significant Gravitas, 2023] and BabyAGI [Nakajima, 2023] implement multi-step autonomous agents without budget controls or code isolation; our layers add both.

Software engineering benchmarks. SWE-bench [Jimenez et al., 2024] evaluates agents on real-world GitHub issues drawn from popular Python repositories: given the codebase and the issue description, the agent must produce a patch, which often spans several functions, classes, or files. HumanEval [Chen et al., 2021] and MBPP [Austin et al., 2021] evaluate function-level code generation from docstrings. LiveCodeBench [Jain et al., 2024] addresses benchmark contamination by continuously collecting fresh competition problems from LeetCode, AtCoder, and CodeForces. These benchmarks focus on isolated coding problems; we target the broader workflow of multi-file, multi-step software engineering tasks that require tool use, testing, and version control.

Agent frameworks and orchestration. LangChain [LangChain, 2022] provides modular abstractions for building LLM applications, including agent loops, tool integration, and memory. It focuses on composability and breadth of integrations and leaves the concerns of software development to the application. DSPy [Khatab et al., 2024] treats LLM calls as declarative modules whose prompts and few-shot examples are compiled and optimized automatically, enabling systematic improvement of multi-stage pipelines; it optimizes prompts and does not run software engineering workflows. CrewAI [CrewAI, Inc., 2024] orchestrates role-playing autonomous agents that collaborate through configurable process models and event-driven flows, emphasizing multi-agent coordination over the layered safety and budget controls that we prioritize. smolagents [Roucher et al., 2025], Hugging Face’s minimalist agent library, has its CodeAgent write actions as executable Python snippets, reducing step counts by 30%. The OpenAI Agents SDK [OpenAI, 2025c] offers a provider-agnostic framework for multi-agent workflows with handoffs, guardrails, and tracing, while Google’s Agent Development Kit [Google, 2025b] provides an open-source toolkit for building, evaluating, and deploying AI agents with multi-agent orchestration. Both provide general-purpose agent infrastructure and leave domain-specific concerns to the application layer.

LLM agent surveys. Several surveys map the design space of LLM-based agents. Xi et al. [2023] surveys the design space of LLM agents along three dimensions, brain (reasoning), perception (input modalities), and action (tool use), and catalogs applications across social science, natural science, and engineering. Wang et al. [2024] propose a systematic framework for autonomous agents built on LLMs, covering profile, memory, planning, and action modules.

IDE integration. GitHub Copilot [GitHub, 2021], Cursor [Anysphere, Inc., 2026], and Windsurf [Codeium, 2024] provide AI assistance within editors. We operate at the level of autonomous multi-step task execution with full tool access and git-level isolation.

6.1 Dynamic Steering of Running Agents

Appendix B.2 describes KISS Sorcar’s dynamic steering: a follow-up user message enters a running task’s conversation before the next model step, uniformly across all surfaces and backends. Several recent systems provide related forms of in-progress control. Claude Code’s Remote Control allows a user to continue a local session from a browser or phone, keep the conversation synchronized across devices, and send messages interchangeably from terminal, web, and mobile surfaces [Anthropic, 2026c]. KISS Sorcar differs in scope: dynamic steering is an IDE/runtime primitive for any KISS-managed agent, including third-party backends and parallel sub-agents; Remote Control is a remote surface for one vendor’s agent. GitHub Copilot’s cloud agent can be monitored and steered with follow-up prompts in GitHub, and users can mention `@copilot` on a pull request to request further changes [GitHub, 2026]. Issue comments added after assignment are not seen until the pull request exists; KISS Sorcar treats steering as a live message to the active task.

Cursor can launch cloud agents from web, desktop, Slack, GitHub/Bitbucket comments, Linear, and an API, and it provides artifacts plus remote desktop control of the agent’s environment [Cursor, 2026c]. Codex has added mobile remote workflows, queue-versus-steer follow-up behavior, side chats, and progress visibility for running threads and subagents [OpenAI, 2026a], and Cursor’s agent accepts a follow-up that is delivered at the agent’s next tool call, or queued for after the turn [Cursor, 2026a]. KISS Sorcar routes the steering message through the same local task machinery as the original instruction, with no cloud session, remote desktop, or PR comment loop.

Other related mechanisms are more specialized. Aider’s `-watch-files` mode lets users place AI! or AI? comments in source files, which Aider collects as coding instructions from the editor [Gauthier, 2026]. LangChain/LangGraph’s human-in-the-loop middleware pauses execution around configured tool calls and resumes after a human approves, edits, rejects, or responds to the pending action [LangChain, 2026]. Semantic Kernel similarly describes the agent loop as repeated function calling until completion or until the model needs user help [Microsoft, 2025]. These frameworks provide useful interrupt/resume substrates, but they center on tool-call approval or model-requested help. Dynamic steering in KISS Sorcar is broader: the user may inject revised task guidance even when the agent did not ask for input.

Recent advances in agentic software engineering. Hassan et al. [2025] articulate the foundational pillars of *Agentic Software Engineering* (SE 3.0), defining a research roadmap that emphasizes trust, controllability, and goal-oriented task decomposition. Li et al. [2025] surveys autonomous coding agents and characterizes the transition from assistive code completion to full-fledged AI teammates that initiate, plan, and execute development tasks. Budget tracking, step limits, and worktree isolation are instances of the controllability and isolation they call for.

Wang et al. [2025a] surveys AI agentic programming techniques, cataloging how LLM-based agents decompose goals, interact with compilers and version control systems, and self-correct through feedback loops. Chatlatanagulchai et al. [2025] characterize 2,303 *agent context files* (persistent, project-level instruction files that guide agentic coding tools) from 1,925 repositories and catalog what they contain, without measuring their effect on agent performance. Our system prompt and the SORCAR.md override are files of this kind. Mohsenimofidi et al. [2025] study the AGENTS.md files of 466 open-source projects and find no established content structure; the study is descriptive and does not measure the files’ effect on agents.

Evolving benchmarks for coding agents. SWE-bench Pro [Deng et al., 2025] introduces a substantially more challenging benchmark with 1,865 long-horizon problems drawn from 41 repositories, including commercial codebases, explicitly targeting enterprise-level complexity beyond the original SWE-bench. These long-horizon tasks (often requiring multi-file patches and hours of professional

developer effort) directly motivate our continuation mechanism. Prathifkumar et al. [2025] test for contamination by asking two Claude models to name the files an issue’s fix edits from the issue text alone; the models localize SWE-Bench Verified files far more often than files of BeetleBox and SWE-rebench issues, so part of a Verified score may be memory. Horikawa et al. [2025] study 15,451 refactorings in 12,256 agent-authored pull requests and find them dominated by low-level consistency edits (renames, extractions, import cleanups).

Self-improvement and test-time scaling. Robeyns et al. [2025] demonstrates a self-improving coding agent that iteratively refines its own scaffolding code and improves its benchmark scores through self-generated changes. Our agent does not rewrite its own scaffolding; the semantic memory of Section 2.3 is the one thing it carries between tasks, and its protocol requires every page to cite the file path or URL that lets the fact be re-verified and asks the agent to delete pages that turn out to be wrong. Gao et al. [2025] introduce the Trae Agent, which spends test-time compute on ensemble reasoning: generation agents produce candidate patches, pruning agents discard the weak ones, and selection agents pick the final patch, reaching 75.20% Pass@1 on SWE-bench Verified. We spend extra compute differently: one developer, a read-only reviewer from a second vendor, and a hard budget ceiling within which the Relentless Agent continues unfinished work across sub-sessions.

Community-driven prompt engineering. Get Shit Done (GSD) [TÂCHES, 2025] is a light-weight meta-prompting, context engineering, and spec-driven development system originally created for Claude Code and later extended to other AI coding agents. GSD addresses *context rot* (the quality degradation that occurs as an LLM fills its context window) by structuring work into phased planning documents and spawning specialized parallel agents for research, planning, execution, and verification. The “Pre-flight Checks”, “Deep Work”, and “Pre-Finish Verification” of `SYSTEM.md` were partly inspired by this work.

Work from the last four months. Several results published between July and September 2026 bear on the design choices of this paper. On evaluation, Liu et al. [2026] audit 254 SWE-bench submissions and find that the leaderboard can no longer order its top entries: the two leading Verified entries each resolve 396 of 500 instances, the top ten share 285 successes and 51 failures, paired McNemar tests separate none of the 29 adjacent top-thirty pairs, and within-model scaffold ranges (29.8 points) exceed the spread of the top thirty (8.8 points). Our own benchmark comparison (Section 5) therefore holds the tasks and the models fixed, changes the harness (date, effort, price list, cap, and prompt also differ, as that section lists), and reports intervals over tasks; the case studies of Section 4 report released artifacts with their audit trails. Li et al. [2026] move in the same direction with Long-Horizon-Terminal-Bench, which grades intermediate progress on terminal tasks that need hours and hundreds of steps, where a single final outcome tells little. Kraishan [2026] study 37,623 provenance-labeled pull requests from five commercial agents and find vendor-specific quality differences (Codex pull requests reverted at 6.1% versus 11.5% for humans, Devin at 14.5%), which supports letting the developer model vary from task to task, as `set_model` does.

On harness design, Xu et al. [2026] compare six tool architectures over 11,700 trajectories and show that how capabilities are exposed changes agent behavior even when the capabilities are the same: structured low-level interfaces improve consistency across repeated attempts by up to $4.7\times$, and Python CodeAct-style interfaces reach similar task performance with 41.6% fewer steps and 56.3% less token usage than a bash-only agent, while lightweight “record your reasoning” tools have little effect. Our tool layer sits at the structured-low-level point of that design space (six file and shell tools, four of them with the Claude Code names, plus browser, memory, decision, and dispatch tools). Lulla et al. [2026] name the level above harness configuration *loop engineering*: systems that start agent runs on a schedule or on repository events and stop them when a machine-checkable condition holds, with persistent state files, verifier sub-agents, token budgets, and defined escalation points; practitioners began using the term in June 2026, and in their mining study of 36,710 repositories, 217 of the 256 heuristic matches were confirmed loops. The single-prompt drivers of Appendix A, the cron agent, and the inbound channel gateways are loop engineering in this sense, and the ideas ledger and `./tmp/PROGRESS.md` are the persistent state files that their gray-literature review prescribes and that, in their mining study, almost no repository commits. Dai and Wang [2026] learn instruction-level execution guardrails from 642 failure traces of coding agents (agents that modify unrelated files, rewrite tests, or ignore failed validations), cutting the abnormal-execution rate of Claude Code with Claude Haiku 4.5 from 69.0% to 26.7%; the testing and pre-flight rules of `SYSTEM.md` are hand-written guardrails against the same failure classes. Goswami [2026] formalize which subset of existing evidence (tests, type checks, proofs, traces) a change must re-establish as a minimum-cost

assurance envelope solved with CP-SAT; our “run only the impacted tests, run the full suite once” rule is an informal version of the same selection problem. Finally, Xiong and Zhang [2026] measure the OpenClaw agent-skill registry after its 2026 boom: 85% of readable skills carry privilege evidence, about 78% have zero stars and zero comments, and three security scanners disagree on 23,702 of 61,990 skills. KISS Sorcar loads Agent Skills from the same directory conventions; on Linux, the 24 Muse-supported connectors of Appendix B.6 leave only surrogate tokens in the agent process, so a skill running in that process never sees a real credential for them.

7 Limitations

Section 3.4 lists the limits of the task database and the ablation. The case studies are two single runs, one per procedure, with no rerun, no controlled baseline agent or person on the same task (HydraKV’s starting engine came from one earlier Sorcar task with no adversarial workloads), and no ablation of the procedure. Only the Bespoke OLAP task ran the seven-step loop, without its search steps; HydraKV ran the earlier paragraph and never used the two log files. Neither task was run without the procedure or without review, and Terminal-Bench 2.0 does not test the procedures. HydraKV’s speedup is against a figure in its task text for the scored mix only, its 10 Mops/s goal was not met, and its held-out mix ran 29% slower than the scored trace. The Bespoke OLAP speedup is against a single-threaded artifact and carries the other caveats of Section 4.2. In both runs the second-vendor reviewer is also a stronger model than the developer, so nothing here separates “different vendor” from “stronger model” or from a second pass of any kind. The author of the task prompts is one of us. The Terminal-Bench 2.0 comparison with the study’s published numbers differs from its protocol in date, effort, price list, cap, and prompt at once; the paired run removes all but effort and prompt, for Pi only, on one model. Memory, the channel agents, and the features of Appendix B are not evaluated.

8 Conclusion

A layered, single-concern agent framework of 4,531 lines, with a system prompt that states engineering rules and two long-run procedures as instructions, was enough to build KISS Sorcar and, through it, two systems artifacts. Six months of the agent’s own task database show the mechanisms in use at scale: 773 tasks continued across context windows, 400 steered mid-flight, 0.6% of 28,832 edits made blind, 11 mock-based test files out of 1,376, and 4,112 review sub-tasks, of which the 1,675 read-only ones with a persisted result report a defect 71% of the time. A controlled ablation on 10 small tasks shows that the prompt’s rules change how the agent works and what it leaves behind, and not whether the hidden tests pass; on tasks of that size, they all pass regardless. A controlled comparison shows that a read-only reviewer from a second vendor reports at least one real defect the developer model’s tests missed in 15 of 20 runs, that 15 of its 17 real findings reproduce on artifacts built without a reviewer (and 4 of 21 findings were not defects), and that the review costs $2.3\times$ the cell. Two long runs produced a key-value store that sustains 5.50 Mops/s on the scored YCSB-A mix over 22.6 agent hours, under an earlier discovery paragraph and an adversarial task prompt, and a TPC-H engine $34.5\times$ faster than its single-threaded artifact, under the seven-step loop without its web-search steps; in both, the defects that mattered were found by a second vendor’s reviewer or an external auditor after the developer model’s own tests had passed. On the HarnessTax study’s 30 Terminal-Bench 2.0 tasks, without the procedures, the agent solved 75.6% of attempts (74.1% with solves past 100 turns counted as failures) against 70.0% for the best published harness, a success gap within sampling error; the paired gap over Pi rerun on one model is +11.1 points ($p = 0.016$). The open question the ablation leaves is whether the rules that were invisible on small tasks are what made the large ones possible; answering it needs long tasks run under controlled conditions, and the harness we release is a start.

Reproducibility statement

The repository contains the framework source, the ablation harness (papers/kissorcar/ablation/: ten tasks with fixtures, hidden tests, and reference solutions; `arms.py`, `run_one.py`, `run_all.py`, `analyze.py`; the 120 cell records with every logged tool call; the probes of Section 3.3), the database mining, case-study mining, and review-classification scripts with their JSON outputs (papers/kissorcar/evidence/), the Terminal-Bench 2.0 benchmark agent with the archived text of its system prompt and the per-attempt records of all three

Terminal-Bench 2.0 runs, the scripts that recompute the code, case-study, and benchmark macros and draw Figure 4 (`ks_numbers.py`, `ks_tb2.py`), and both case-study projects under `projects/` with their tests, discovery logs, audit reports, and raw logs. Every macro in the source names, in a comment, the script or JSON key it is read from. The task database itself contains private prompts and is not released; the mining scripts run on any `sorcar.db`. Rerunning the ablation needs API keys for Anthropic and OpenAI and cost about \$127 in total; rerunning the Terminal-Bench 2.0 runs needs Anthropic, OpenAI, and OpenRouter keys, and the 30-task run cost \$649, the run on the other 59 tasks \$270, and the Pi run \$130.

AI use statement

The system described here wrote its own code under the prompts recorded in its task database, including the Terminal-Bench 2.0 benchmark agent and the scripts that compute the numbers in this paper. The text of this paper was drafted and revised with the same agent from the authors’ outlines and edits, and every number in it is a macro whose comment names the script, file, or task record that produces it; a read-only review by a second model was used on the paper as on the code. We take responsibility for all content.

Acknowledgments

This research is supported in part by gifts from Accenture, Amazon, AMD, Anyscale, Broadcom, Google, IBM, Intel, Intesa Sanpaolo, Lambda, Lightspeed, Mibura, NVIDIA, Samsung SDS, SAP, by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research through the X-STACK: Programming Environments for Scientific Computing program (DE-SC0021982) and the Defense Advanced Research Projects Agency (DARPA) under Agreement No. HR00112590134.

We would like to thank Marius Momeu for finding critical bugs in the cost computation and in the UI usability, Yogya Mehrotra for testing and fixing bugs in the OpenClaw like `third_party_agents` in KISS Sorcar; Debabrata Dash, Yiwei Hou, Kaiyao Ke, Muxi Lyu, Anoop Mishra, Manish Shetty, Ion Stoica, Shangyin Tan, Hao Wang, and Matei Zaharia for various useful feedback.

References

- Nikita Agarwal and Nivedit Jain. FIRE: Failure-informed runtime engineering for reliable language-model agents. arXiv preprint arXiv:2609.26048, 2026.
- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2026. Oral. arXiv preprint arXiv:2507.19457.
- Algorithmic Superintelligence. OpenEvolve: Open-source implementation of AlphaEvolve. <https://github.com/algorithmicsuperintelligence/openevolve>, 2025.
- Anthropic. Anthropic prompt engineering guide. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview>, 2025a.
- Anthropic. Claude 3.7 Sonnet and Claude Code. <https://www.anthropic.com/news/claude-3-7-sonnet>, February 2025b. Announcement of Claude Code, 24 February 2025. Product page: <https://claude.com/product/claude-code>.
- Anthropic. Introducing Claude Opus 4.6. <https://www.anthropic.com/news/claude-opus-4-6>, 2026a.
- Anthropic. Introducing Claude Opus 4.7. <https://www.anthropic.com/news/claude-opus-4-7>, 2026b.
- Anthropic. Claude Code Remote Control: Continue local sessions from any device. <https://docs.anthropic.com/en/docs/claude-code/remote-control>, 2026c.
- Anthropic. Claude Code documentation: Explore the context window. <https://code.claude.com/docs/en/context-window>, 2026d. Accessed 18 September 2026.

- Anthropic. Claude Code documentation: Extend Claude with skills. <https://code.claude.com/docs/en/skills>, 2026e. Accessed 27 September 2026.
- Anthropic. Claude Code documentation: Common workflows (scheduling, worktrees, subagents). <https://code.claude.com/docs/en/common-workflows>, 2026f. Accessed 18 September 2026.
- Anysphere, Inc. Cursor: AI coding agent for building ambitious software. <https://cursor.com>, 2026. Accessed September 2026.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Oussama Ben Sghaier, Hao Li, Bram Adams, and Ahmed E. Hassan. Don’t blame the large language model: How agent harness evolution shapes coding agent quality. *arXiv preprint arXiv:2607.03691*, 2026.
- Nicholas Carlini. Building a C compiler with a team of parallel Claudes. Anthropic engineering blog, <https://www.anthropic.com/engineering/building-c-compiler>, February 2026.
- Badrish Chandramouli, Guna Prasaad, Donald Kossmann, Justin Levandoski, James Hunter, and Mike Barnett. FASTER: A concurrent key-value store with in-place updates. In *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, pages 275–290. ACM, 2018. doi: 10.1145/3183713.3196898.
- Worawalan Chatlatanagulchai, Hao Li, Yutaro Kashiwa, Brittany Reid, Kundjanasith Thonglek, Pattara Lee-laprute, Arnon Rungsawang, Bundit Manaskasemsak, Bram Adams, Ahmed E. Hassan, and Hajimu Iida. Agent READMEs: An empirical study of context files for agentic coding. *arXiv preprint arXiv:2511.12884*, 2025.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Audrey Cheng, Shu Liu, Melissa Pan, Zhifei Li, Bowen Wang, Alex Krentsel, Tian Xia, Mert Cemri, Jongseok Park, Shuo Yang, Jeff Chen, Lakshya Agrawal, Aditya Desai, Jiarong Xing, Koushik Sen, Matei Zaharia, and Ion Stoica. Barbarians at the gate: How AI is upending systems research. *arXiv preprint arXiv:2510.06189*, 2025.
- Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP)*, pages 268–279, 2000. doi: 10.1145/351240.351266.
- Codeium. Windsurf: The AI-powered IDE. <https://windsurf.com>, 2024.
- Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC)*, pages 143–154. ACM, 2010. doi: 10.1145/1807128.1807152.
- CrewAI, Inc. CrewAI: Framework for orchestrating role-playing, autonomous AI agents. <https://github.com/crewAIInc/crewAI>, 2024.
- Cursor. Cursor documentation: Agent overview, sections “queued messages” and “steer a running agent”. <https://cursor.com/docs/agent/overview>, 2026a. Accessed 26 September 2026.
- Cursor. Cursor documentation: Automations, agent overview, and bring your own API key. <https://cursor.com/docs/cloud-agent/automations>, 2026b. Accessed 18 September 2026.
- Cursor. Cursor cloud agents. <https://cursor.com/docs/cloud-agent>, 2026c.
- Cursor. Cursor help: Skills. <https://cursor.com/help/customization/skills>, 2026d. Accessed 27 September 2026.
- Cursor Research. Composer 2 technical report. *arXiv preprint arXiv:2603.24477*, 2026.

- Cursor Team. Introducing Cursor 2.0 and Composer. <https://cursor.com/blog/2-0>, October 2025.
- Wuyang Dai and Song Wang. AgentGuard: Learning execution guardrails from anomalous coding-agent trajectories. *arXiv preprint arXiv:2609.16287*, 2026.
- Xiang Deng, Jeff Da, Edwin Pan, Yannis Yiming He, Charles Ide, Kanak Garg, Niklas Lauffer, Andrew Park, Nitin Pasari, Chetan Rane, et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.
- Earendil Works. Pi: A minimal, extensible AI agent for the terminal. <https://github.com/earendil-works/pi>, 2026. Package @earendil-works/pi-coding-agent. Accessed 25 September 2026.
- Gil Einziger, Roy Friedman, and Ben Manes. TinyLFU: A highly efficient cache admission policy. *ACM Transactions on Storage*, 13(4):35:1–35:31, 2017. doi: 10.1145/3149371.
- Run-Ze Fan, Zihao Zhang, Simin Ma, Yebowen Hu, Shouju Wang, Kaiqiang Song, Fei Liu, Hamed Zamani, and Xiaoyang Wang. An empirical study of harness design for coding agents. *arXiv preprint arXiv:2609.20804*, 2026.
- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Prompt-breeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- Pengfei Gao, Zhao Tian, Xiangxin Meng, and Trae Research Team. Trae agent: An LLM-based agent for software engineering with test-time scaling. *arXiv preprint arXiv:2507.23370*, 2025.
- Paul Gauthier. Aider: AI pair programming in your terminal. <https://github.com/paul-gauthier/aider>, 2023.
- Paul Gauthier. Aider in your IDE: AI comments and file watching. <https://aider.chat/docs/usage/watch.html>, 2026.
- GitHub. GitHub Copilot: Your AI pair programmer. <https://github.com/features/copilot>, 2021.
- GitHub. Using Copilot cloud agent on GitHub. <https://docs.github.com/en/copilot/how-tos/use-copilot-agents/cloud-agent/use-cloud-agent-on-github>, 2026.
- Google. Gemini prompt engineering strategies. <https://ai.google.dev/gemini-api/docs/prompting-strategies>, 2025a.
- Google. Agent Development Kit (ADK): An open-source framework for building AI agents. <https://google.github.io/adk-docs/>, 2025b.
- Google DeepMind. Gemini 3.1 Pro. <https://deepmind.google/models/gemini/pro/>, 2026.
- Anjan Goswami. Assurance envelopes for autonomous coding agents: Minimum-cost evidence for software change. *arXiv preprint arXiv:2609.16302*, 2026.
- Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yun Xiong, and Wenfeng Liang. DeepSeek-Coder: When the large language model meets programming — the rise of code intelligence. *arXiv preprint arXiv:2401.14196*, 2024.
- Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments. <https://github.com/harbor-framework/harbor>, 2026. Version 0.20.0 used here; DOI 10.5281/zenodo.20953922.
- Ahmed E. Hassan, Hao Li, Dayi Lin, Bram Adams, Tse-Hsun Chen, Yutaro Kashiwa, and Dong Qiu. Agentic software engineering: Foundational pillars and a research roadmap. *arXiv preprint arXiv:2509.06216*, 2025.
- Christian Holler, Kim Herzig, and Andreas Zeller. Fuzzing with code fragments. In *Proceedings of the 21st USENIX Security Symposium*, 2012.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations (ICLR)*, 2024.
- Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida, and Ahmed E. Hassan. Agentic refactoring: An empirical study of AI coding agents. *arXiv preprint arXiv:2511.04824*, 2025.

- Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01798.
- Geoffrey Huntley. Ralph Wiggum as a “software engineer”. Blog post, <https://ghuntley.com/ralph/>, July 2025.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Zhengyao Jiang, Dominik Schmidt, Dhruv Srikanth, Dixing Xu, Ian Kaplan, Deniss Jacenko, and Yuxiang Wu. AIDE: AI-driven exploration in the space of code. arXiv:2502.13138, 2025.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL <https://openreview.net/forum?id=VTF8yNQM66>. arXiv:2310.06770.
- R. Karedla, J. S. Love, and B. G. Wherry. Caching strategies to improve disk system performance. *Computer*, 27(3):38–46, March 1994. doi: 10.1109/2.268884.
- Andrej Karpathy. autoresearch: AI agents running research on single-GPU nanochat training automatically. <https://github.com/karpathy/autoresearch>, March 2026.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling declarative language model calls into self-improving pipelines. In *The Twelfth International Conference on Learning Representations*, 2024.
- Minseo Kim. tap: A file-based protocol for heterogeneous LLM agent collaboration. arXiv preprint arXiv:2606.14445, 2026. Accepted to KCC 2026; 3 pages.
- Kimi Team. Kimi K2.5: Visual agentic intelligence. *arXiv preprint arXiv:2602.02276*, 2026.
- Obada Kraishan. Not all agents are equal: Code quality and post-merge maintenance across five autonomous coding agents in the wild. *arXiv preprint arXiv:2609.17598*, 2026.
- LangChain. LangChain: Build context-aware reasoning applications. <https://github.com/langchain-ai/langchain>, 2022.
- LangChain. Human-in-the-loop middleware. <https://docs.langchain.com/oss/python/langchain/human-in-the-loop>, 2026.
- Robert Tjarko Lange, Yuki Imajuku, and Edoardo Cetin. ShinkaEvolve: Towards open-ended and sample-efficient program evolution. *arXiv preprint arXiv:2509.19349*, 2025.
- Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. The rise of AI teammates in software engineering (SE 3.0): How autonomous coding agents are reshaping software engineering. *arXiv preprint arXiv:2507.15003*, 2025.
- Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. StarCoder: May the source be with you! *Transactions on Machine Learning Research (TMLR)*, 2023.
- Zongxia Li, Zhongzhi Li, Yucheng Shi, Ruhan Wang, Junyao Yang, Zhichao Liu, Xiyang Wu, Anhao Li, Yue Yu, Ninghao Liu, Lichao Sun, Haotao Mi, and Leowei Liang. Long-horizon-terminal-bench: Testing the limits of agents on long-horizon terminal tasks with dense reward-based grading. *arXiv preprint arXiv:2607.08964*, 2026.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. arXiv preprint arXiv:2604.25850, 2026.
- Fengshuo Liu, Ying Liu, Ruize Sun, Lie Luo, and Siyuan Guo. Coding agents have converged: Why the SWE-bench leaderboard can no longer order its top entries, and what to measure instead. *arXiv preprint arXiv:2609.17394*, 2026. Accepted at ADMA 2026.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist: Towards fully automated open-ended scientific discovery. arXiv:2408.06292, 2024.

- Jai Lal Lulla, Vahram Nersesyan, Seyedmoein Mohsenimofidi, Christoph Treude, and Sebastian Baltes. Loop engineering: Building blocks, adoption, and impact. *arXiv preprint arXiv:2608.21884*, 2026.
- Yanuo Ma, Ben Kereopa-Yorke, and Ben Schultz. Building to the test: Coding agents deliver what you check, not what you requested. *arXiv preprint arXiv:2606.28430*, 2026.
- William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.
- Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.
- Microsoft. Planning in Semantic Kernel. <https://learn.microsoft.com/en-us/semantic-kernel/concepts/planning>, 2025.
- Seyedmoein Mohsenimofidi, Matthias Galster, Christoph Treude, and Sebastian Baltes. Context engineering for AI agents in open-source software. *arXiv preprint arXiv:2510.21413*, 2025.
- Yohei Nakajima. BabyAGI. <https://github.com/yoheinakajima/babyagi>, 2023.
- Alexander Novikov, Ngân Vŭ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*, 2025.
- OpenAI. Introducing Codex. <https://openai.com/index/introducing-codex/>, May 2025a. A cloud-based software engineering agent powered by codex-1. Blog post, 16 May 2025; open-source CLI at <https://github.com/openai/codex>.
- OpenAI. Prompt engineering best practices. <https://platform.openai.com/docs/guides/prompt-engineering>, 2025b.
- OpenAI. OpenAI Agents SDK: A lightweight, powerful framework for multi-agent workflows. <https://github.com/openai/openai-agents-python>, 2025c.
- OpenAI. Codex changelog. <https://developers.openai.com/codex/changelog/>, 2026a.
- OpenAI. Introducing GPT-5.5. <https://openai.com/index/introducing-gpt-5-5/>, 2026b.
- OpenClaw AI. OpenClaw: Personal AI assistant. <https://openclaw.ai>, 2025.
- OpenHands. Condenser: High-level architecture of the conversation history compression system (LLMSummarizingCondenser), OpenHands software agent SDK documentation. <https://docs.openhands.dev/sdk/arch/condenser>, 2026. Accessed 18 September 2026.
- Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024. *arXiv preprint arXiv:2406.11695*.
- Anne Ouyang, Simon Guo, Simran Arora, Alex L. Zhang, William Hu, Christopher Ré, and Azalia Mirhoseini. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2502.10517*, 2025.
- Melissa Z. Pan, Shuo Yang, Negar Arabzadeh, Wei-Lin Chiang, Ion Stoica, and Matei Zaharia. HarnessTax: How much does the harness matter for coding agents? <https://harnesstax.github.io/>, 2026. Blog post with per-model tables; 21 model-harness pairs on Terminal-Bench 2.0 and SWE-bench Lite. Accessed 25 September 2026.
- Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. doi: 10.52202/079017-2197.
- Sungho Park, Wonjoong Kim, Rongyuan Tan, Jue Zhang, Wook-Shin Han, Pengfei Gao, Chanyoung Park, Yongqiang Yao, Rao Fu, Elsie Nallipogu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. AutoSaddler: Automatic harness optimization with durable updates from agent execution traces. *arXiv preprint arXiv:2608.23041*, 2026.
- Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3419–3448, 2022. doi: 10.18653/v1/2022.emnlp-main.225.

- Thanosan Prathifkumar, Noble Saji Mathews, and Meiyappan Nagappan. Does SWE-Bench-Verified test agent ability or model memory? *arXiv preprint arXiv:2512.10218*, 2025.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with “gradient descent” and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. arXiv preprint arXiv:2305.03495.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- Ziheng Qin, Yaxin Lu, Zhangyang Atlas Wang, and Kai Wang. StateM: Reaching 95.3% raw accuracy, or a \$15 frontier run, on Terminal-Bench 2.1 via harness scaling. arXiv preprint arXiv:2608.15089, 2026.
- Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024. doi: 10.1038/s41586-023-06924-6.
- Aymeric Roucher, Albert Villanova del Moral, Thomas Wolf, Leandro von Werra, and Erik Kaunismäki. smolagents: A smol library to build great agentic systems. <https://github.com/huggingface/smolagents>, 2025.
- Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.
- Siddharth Sambharia. The harness tax: The dead weight inside your coding agent. <https://portkey.ai/blog/the-harness-tax/>, April 2026. Blog post, 13 April 2026.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, pages 8634–8652, 2023. doi: 10.52202/075280-0377.
- Significant Gravitas. AutoGPT. <https://github.com/Significant-Gravitas/AutoGPT>, 2023.
- Davide Spadini, Mauricio Aniche, Magiel Bruntink, and Alberto Bacchelli. To mock or not to mock? an empirical study on mocking practices. In *2017 IEEE/ACM 14th International Conference on Mining Software Repositories (MSR)*, pages 402–412, 2017. doi: 10.1109/MSR.2017.61.
- SWE-agent team. mini-SWE-agent: The 100 line AI agent that solves GitHub issues. GitHub repository, <https://github.com/SWE-agent/mini-swe-agent>, v2.4.6, 2026. Accessed 19 September 2026.
- TÂCHES. Get Shit Done: A light-weight meta-prompting, context engineering and spec-driven development system for AI coding agents. <https://github.com/gsd-build/get-shit-done>, 2025. Initial commit December 2025.
- Huanting Wang, Jingzhi Gong, Huawei Zhang, Jie Xu, and Zheng Wang. AI agentic programming: A survey of techniques, challenges, and opportunities. *arXiv preprint arXiv:2508.11126*, 2025a.
- Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*, 2025b. arXiv:2407.16741.

- Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents. *arXiv preprint arXiv:2607.12227*, 2026.
- Johannes Wehrstein, Timo Eckmann, Matthias Jasny, and Carsten Binnig. Bespoke OLAP: Synthesizing workload-specific one-size-fits-one database engines. *arXiv preprint arXiv:2603.02001*, 2026. Accepted to VLDB 2026.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Scott Wu. Introducing Devin, the first AI software engineer. Cognition blog, <https://cognition.com/blog/introducing-devin>, March 2024. Product page: <https://devin.ai>.
- Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, et al. The rise and potential of large language model based agents: A survey. *arXiv preprint arXiv:2309.07864*, 2023.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.
- Yunpeng Xiong and Ting Zhang. After the party: Growth, governance, and security scanning in the OpenClaw agent skill ecosystem. *arXiv preprint arXiv:2609.17274 (to appear at APSEC 2026)*, 2026.
- Xiangzhe Xu, Hamidreza Saghir, Qianhui Wu, Marc-Alexandre Côté, Tong Wang, Kiran Lakkaraju, Kexin Pei, and Xiangyu Zhang. The devil is in the interface: Evaluating how tool architecture shapes coding agent behavior. *arXiv preprint arXiv:2608.11386*, 2026.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*, 2024a. *arXiv preprint arXiv:2309.03409*.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024b. doi: 10.52202/079017-1601. *arXiv:2405.15793*.
- Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. Finding and understanding bugs in C compilers. In *Proceedings of the 32nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 283–294, 2011.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023b. *arXiv:2210.03629*.
- Justin Young. Effective harnesses for long-running agents. Anthropic engineering blog, <https://www.anthropic.com/engineering/effective-harnesses-for-long-running-agents>, November 2025.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. TextGrad: Automatic “differentiation” via text. *arXiv preprint arXiv:2406.07496*, 2024.
- Z.ai. GLM-5.1: Towards long-horizon tasks. Technical blog, <https://z.ai/blog/glm-5.1>, 2026.
- Xing Zhang, Guanghui Wang, Yanwei Cui, Wei Qiu, Ziyuan Li, Bing Zhu, and Peiyang He. Guardrails beat guidance: A large-scale study of rules, skills, and persistent configuration for coding agents. *arXiv preprint arXiv:2604.11088*, 2026.

The appendices describe the parts of KISS Sorcar that the main text refers to but does not need: the single-prompt drivers for AI discovery, adversarial testing, GEPA prompt optimization, and repository optimization (Appendix A); the user-facing features of the three surfaces (Appendix B); the coding and testing blocks of the system prompt with commentary (Appendix C); two development sessions that predate the task database, checked against the commit history (Appendix D); and the ablation tasks and arms (Appendix E).

A Single-Prompt Drivers: AI Discovery, Adversarial Testing, GEPA Prompt Optimization, and Repository Optimization

Beyond per-task coding assistance, KISS Sorcar can be driven as an autonomous *research and optimization driver*: the user issues a single high-level objective with explicit numeric stopping conditions (a target accuracy, a target latency, a budget cap, a “do not stop until...” clause), and the agent then runs an open-ended exploration loop (spawning experiments, reading logs, mutating its own prompts or code, journaling what worked and what failed) until the targets are met or the budget is exhausted. None of the capabilities described in Section 2 was designed for this use case; together they cover what an open-ended driver needs: the Relentless Agent’s structured-summary continuation (Section 2.2) lets multi-hour or multi-day loops survive context exhaustion; the Sorcar Agent’s parallel sub-agents and streamable shell executor (Section 2.3) parallelize the inner experiments and monitor long-running commands in real time; the `set_model` tool lets a cheap model do the routine bookkeeping while a stronger model is engaged only at decision points; the Chat Sorcar Agent persists every task and its result to `sorcar.db`, and every tool call when the task runs in a chat webview, providing a replayable record for self-reflection (Section 2.4); and the Worktree Sorcar Agent’s branch-per-task isolation (Section 2.5) keeps a task’s edits on a branch that can be merged or discarded as a whole.

We ship multiple sample task templates in `src/kiss/SAMPLE_TASKS.md` that exercise this pattern. Each is a single natural-language prompt that fits in a chat box; the agent does the rest. The AI-discovery block of the system prompt supplies the loop the templates rely on (Algorithm 1); a user can add the adversarial-testing procedure of Appendix A.2 by naming it in the task. (A companion “adversarial training” block applies the same generate-and-repair loop to a model under training; one task has invoked it and we do not describe it further.)

Recorded use. The task database (Section 3.1) shows which of these have run. The AI-discovery block, or its earlier one-paragraph form (Appendix F), was invoked by name in the HydraKV tasks of Section 4.1 and in repository-optimization tasks on `xxHash` (5,583 steps, \$393), `SQLite` (two tasks, 466 and 612 steps), and `LZ4` (1,386 steps); the phrase “adversarial testing” appears in the prompts of ten top-level tasks other than those that edit this paper, those four among them. The three sample prompts reproduced below have not been run as shipped: the Modal-based AI Discovery prompt and the repository-optimization template have no verbatim run (the Bespoke OLAP task of Section 4.2 ran a separate optimization prompt of its own with a numeric target), and the GEPA prompt has none (several April and May 2026 tasks ran the separate `gepa.py` script now under `src/kiss/agents/obsolete/gepa/`). The paragraphs headed “What the prompt formulation adds” therefore describe what the templates are built to do; they report no measured outcomes.

A.1 AI Discovery

AI-driven discovery casts scientific and algorithmic problem-solving as an evolutionary search over programs: an outer loop maintains a population of candidate solutions (each a complete piece of source code, a prompt, or a model recipe), an LLM proposes new candidates by mutating or recombining selected parents, and an automated evaluator scores every offspring on a task-specific fitness function, with the fittest individuals fed back as parents for the next generation [Romera-Paredes et al., 2024, Novikov et al., 2025, Algorithmic Superintelligence, 2025, Lange et al., 2025, Cheng et al., 2025, Agrawal et al., 2026]. This is the structure of a classical genetic algorithm (selection, mutation, crossover, replacement) except that a large language model conditioned on natural-language critiques of past failures realizes the variation operators (a genetic algorithm uses random bit-flips), and the genotype is human-readable source code, where a genetic algorithm has a fixed-length bitstring. DeepMind’s *FunSearch* [Romera-Paredes et al., 2024] introduced this paradigm by pairing a code-LLM with an island-based evolutionary population and using execution feedback as

fitness, discovering new constructions for the cap-set problem and improved bin-packing heuristics; *AlphaEvolve* [Novikov et al., 2025] scaled the same recipe to full programs across mathematics, hardware design, and Google infrastructure; *OpenEvolve* [Algorithmic Superintelligence, 2025] added MAP-Elites quality-diversity and an artifact side-channel to the open-source community; *ShinkaEvolve* [Lange et al., 2025] improved sample efficiency with parent sampling that balances exploration and exploitation, code-novelty rejection sampling, and a bandit-based LLM-ensemble selection strategy; *ADRS* [Cheng et al., 2025] demonstrated that this evolutionary loop transfers to systems-performance research whenever the evaluator is a runnable workload; and *GEPA* [Agrawal et al., 2026] showed that the same selection–mutation–crossover skeleton optimizes natural-language prompts when fitness is measured by trajectory-level reflection rather than scalar reward. Common to all of these is a contract: a natural-language objective, code or a prompt as the unit of mutation, an automated evaluator as the verifier, a journal of tried ideas, and explicit anti-reward-hacking and generalization clauses. KISS Sorcar’s AI Discovery template inherits this contract, and the framework supplies the primitives needed to instantiate it: budget-aware tool use, Relentless continuation, parallel sub-agents, per-task worktree isolation, replayable trajectories in `sorcar.db`, and dynamic `set_model` switching.

The exact prompt shipped in `src/kiss/SAMPLE_TASKS.md` is reproduced verbatim below; the user fills in only the data path and runs it as a single chat message:

Sorcar for AI Discovery: Can you AI-discover the lightest and fastest AI model that will give the best accuracy and recall on the data at <<path/to/data>> at the lowest price? Use ‘modal’ CLI to train your models on GPUs and evaluate if needed. The total budget for Modal.com is \$ 1,000. Do not STOP until accuracy/recall reaches 99% and the model’s price per query is less than \$0.50. Create a report.

The prompt is short because the discipline it used to spell out has moved into the system prompt. The “AI discovery, auto research, and optimization” block of `SYSTEM.md` is mandatory whenever the agent does AI discovery, auto research, optimization, or AI research; Algorithm 1 gives it as pseudocode. Steps 1–3 run once. The agent profiles the baseline and records its metrics, searches the literature, and writes ideas with their rationale to `./tmp/ideas.md`. The loop then judges the ideas pairwise, implements the winner, evaluates it end to end, and records the idea, its aspect of improvement, and the measured metrics in `./tmp/explored-ideas.md`. An idea that turned out better is kept, and the agent tries composing it with prior winners on different aspects; an idea that turned out worse is marked failed so that it is never retried. The loop stops when the user’s metric goal is met and a held-out or generalization check shows the result is not overfit. Both ledgers are files under `./tmp/`, so they outlive the context window; when the Relentless Agent (Section 2.2) starts a new sub-session, the record of what has been tried is still on disk. Earlier revisions of the sample prompt carried the ledger, the held-out split, and the “experiment on a subset, then extrapolate” advice inline, together with an anti-reward-hacking clause; they were removed from the prompt once the system prompt block above existed, which enforces the first two.

What the prompt formulation adds. Casting AI discovery as a single natural-language prompt, with no bespoke evolutionary harness, has three consequences beyond the clauses shared by all three templates (see “Common pattern” below). There is no infrastructure to set up: FunSearch, AlphaEvolve, OpenEvolve, and ShinkaEvolve each ask the user to configure an island topology, write an evaluator plug-in, or stand up a programs database, whereas here the ideas ledger, the agent’s persistence layer (`sorcar.db`), and worktree isolation provide the same services. The evaluator can move: when the task also asks for adversarial testing (Appendix A.2), one sub-task breaks the current artifact and another repairs it, and the two can alternate, so the evaluator changes as the artifact does, whereas the frameworks above fix their evaluator when the run starts. And candidates can be evaluated in parallel: the Sorcar Agent’s parallel sub-agents evaluate independent candidates on disjoint slices of the data, which is the population-level parallelism of evolutionary frameworks without additional orchestration code.

A.2 Adversarial Testing

Algorithm 1 scores every candidate against an evaluator, and a candidate can score well on a fixed evaluator by fitting its particulars. The engine HydraKV started from, which a single Sorcar task wrote with no adversarial workloads, did this. It exploited the dense key space of the published YCSB trace, validated all 250 million keys, scored 1.85 Mops/s, and could not have run on sparse keys at

Algorithm 1 The “AI discovery, auto research, and optimization” block of SYSTEM.md (version 2026.9.25) as pseudocode. The block is a numbered list of seven steps; the comments give the step numbers. Step 6 of the block ends with “go to step 4”, which is the loop here. Two things are ours: the input line, and the placement of the step-7 stopping test after each evaluation, since the block states the condition without saying where to test it.

Require: a task with a metric goal G and the data, tests, or baseline it names

- 1: read and profile the data, tests, and baseline; record the baseline metrics ▷ step 1
 - 2: search the web for state-of-the-art approaches, papers, repositories, and issues ▷ step 2
 - 3: write ideas and their rationale to ./tmp/ideas.md ▷ step 3
 - 4: **loop**
 - 5: judge the ideas pairwise; let w be the winner ▷ step 4
 - 6: implement w and run a real end-to-end evaluation ▷ step 5
 - 7: log w , its aspect of improvement, and its metrics in ./tmp/explored-ideas.md
 - 8: **if** w is better **then**
 - 9: keep w and try composing it with prior winners on different aspects
 - 10: **else if** w is worse **then**
 - 11: mark w as failed so it is never retried
 - 12: **end if**
 - 13: **if** G is met and a held-out or generalization check proves the result is not overfit **then** ▷ step 7
 - 14: **stop**
 - 15: **end if**
 - 16: search again for fresh ideas not explored before, based on previous experience, excluding the ideas recorded in ./tmp/explored-ideas.md ▷ step 6
 - 17: **end loop**
-

all (Section 4.1). The “Adversarial testing” block of SYSTEM.md is the system prompt’s response, reproduced here in full:

```
## Adversarial testing

Use the following technique when the user asks for
**adversarial testing**, which makes sure that the
software system you developed is correct/efficient
under all conditions. Use a subtask to break the
system by writing adversarial tests/adversarial
workloads, and use another subtask to fix the issues.
```

The block applies when the user’s task contains the phrase “adversarial testing” and assigns two roles to two sub-tasks. The breaker writes tests or workloads that the current artifact was not built against and that make it fail or slow down. The fixer repairs the artifact on the accumulated tests and workloads. “Subtask” is the system prompt’s word for a sub-agent started through the Sorcar Agent’s `run_parallel` tool (Section 2.3); each runs in its own LLM context, seeded with the parent chat’s history, so the fixer’s trajectory does not reach the breaker. The block itself says nothing about repetition. HydraKV’s adversarial discovery task (the second Sorcar task of the project; the first wrote the starting engine) supplied the iteration in its task prompt: generate variants and repair the engine, then measure it on a fresh variant with no further tuning. That run used the earlier discovery paragraph and did not meet its 10 Mops/s goal (Section 4.1); since no repair was made in response to the final variant, it tests whether the repairs overfit the earlier ones, the role of the held-out check in step 7 of Algorithm 1.

The two roles are old in software engineering. Fuzzing [Holler et al., 2012] and property-based testing [Claessen and Hughes, 2000] generate inputs the developer did not anticipate; differential testing [McKeeman, 1998, Yang et al., 2011] uses a second implementation as the oracle, as the Bespoke OLAP case study did with DuckDB (Section 4.2); and red-teaming a language model with a second language model [Perez et al., 2022] has the same two-role shape. The block does not name them; the web-research protocol leads the agent to the ones relevant to the task at hand, as in HydraKV, where the workload variants were grounded in published studies of key-range locality and dynamic popularity. The block adds only the assignment of the two roles to two separate sub-tasks under one long-running task, so that the generate-and-repair loop runs without a human writing the next test.

HydraKV’s discovery task ran this loop from an inline prompt in July 2026, before the block existed; the block entered `SYSTEM.md` in August 2026. The record shows what the loop found (Table 3): a sparse-key variant that OOM-killed the dense index in minutes and forced the switch to a fingerprint hash index (the rebuilt engine later scored 5.67 Mops/s on it), a clustered variant at 5.56 Mops/s, a temporal-drift variant at 3.97 Mops/s that no engine change recovered without hurting the stationary workloads, and a held-out variant generated after the loop closed at 3.87 Mops/s, just below the drift variant.

A.3 GEPA Prompt Optimization

Prompt optimization is the problem of automatically rewriting one or more natural-language prompts inside a (possibly compound) LLM system so that a downstream task metric is maximized, without access to module-level labels or weight gradients [Yang et al., 2024a, Pryzant et al., 2023, Fernando et al., 2023, Opsahl-Ong et al., 2024, Yuksekgonul et al., 2024, Khattab et al., 2024, Agrawal et al., 2026]. The shared template is an outer loop that proposes candidate prompts, evaluates each on a labeled minibatch, and keeps the best for the next round (a *discrete* analogue of stochastic optimization in which the LLM itself is the variation operator). OPRO [Yang et al., 2024a] formalized the loop by passing the LLM a history of past prompts together with their numeric scores and asking for a better one; ProTeGi/APO [Pryzant et al., 2023] replaced scalar rewards with natural-language “gradients” (LLM-written critiques of the current prompt) and propagated them through beam search; Promptbreeder [Fernando et al., 2023] evolved a population of task-prompts together with the mutation prompts that mutate them, in a self-referential genetic algorithm; MIPROv2 [Opsahl-Ong et al., 2024] extended these ideas to multi-stage LM programs in DSPy [Khattab et al., 2024] by jointly searching over instructions and few-shot demonstrations under a stochastic minibatch surrogate; and TextGrad [Yuksekgonul et al., 2024] cast the whole compound system as an autograd-style graph through which textual feedback is backpropagated. GEPA [Agrawal et al., 2026] unifies these threads in a single “Genetic-Pareto” optimizer that (i) samples full trajectories of a system, tool calls, intermediate model responses, and outputs, rather than only final answers, (ii) uses reflective natural-language critiques as mutation operators in the spirit of ProTeGi and TextGrad, (iii) maintains a Pareto frontier of complementary prompts in the spirit of MAP-Elites quality-diversity, and (iv) combines complementary lessons across frontier nodes by crossover. GEPA reports up to 20% absolute gain over GRPO with up to $35\times$ fewer rollouts on six tasks and beats MIPROv2 by over 10%. The Sorcar GEPA template instantiates this algorithm against a ChatSorcarAgent target, with the agent’s per-tool-call trajectory store (`sorcar.db`, Section 2.4) supplying the trajectory data on which reflection operates.

The GEPA prompt, also from `src/kiss/SAMPLE_TASKS.md`, is likewise run as one chat message after the user fills in the data path (the two crossover sub-steps that repeat the mutation branch word for word are shortened here):

Sorcar GEPA Prompt Optimizer: Can you optimize a prompt for a ChatSorcarAgent of the kiss-agent-framework Python library using the following GEPA algorithm on the data at <<url_or_db_file_of_data>> using claude-fable-5? You can find the trajectory events of an agent execution in `~/kiss/sorcar.db` after the agent has finished its execution. Split the dataset into a 50% dev set and a 50% val set.

RUN_GEPA: Sample 100 data points from the val set and call it the sval set. Maintain a Pareto frontier in the folder `./pareto`, with a sub-folder for each node in the frontier. A node contains a prompt file (`prompt.md`) and a JSON file, say `score.json`, containing the list of data points (ids) from the sval set that were correctly predicted by the prompt. When you add a node to the Pareto frontier, make sure that the list of correctly predicted data points is not a subset of or equal to an existing list of data points in some node in the frontier. If such a node exists, do not add the new node. After adding a node, remove all nodes whose list of data points is a subset of or equal to the list of data points in the added node. Then run the following algorithm.

1. *Pick a node from the Pareto frontier with probability 0.5*
 - (a) *sample a minibatch of 5 data points from the dev set*
 - (b) *run the agent with the prompt from the node on the minibatch*
 - (c) *If the agent incorrectly predicts for some data points, analyze and reflect on the trajectory events of the agent on those data points available at `~/kiss/sorcar.db` and propose a new prompt that will fix the mistakes made by the agent on data points incorrectly predicted*

- (d) If the agent predicts correctly on the minibatch, then evaluate it on the sval set and create the list of data points on which the agent with the new prompt predicts correctly.
 - (e) Add the new prompt and the list of data points to the Pareto frontier
 2. Pick two nodes from the Pareto frontier randomly with the remaining probability.
 - (a) Sample a fresh minibatch of 5 dev-set data points
 - (b) Merge the prompts from the two nodes into a new prompt.
 - (c) If the agent predicts correctly on the minibatch with the new prompt, then evaluate it on the sval set and create the list of datapoints on which the agent with the new prompt predicts correctly.
 - (d) Add the merged prompt and its list of data points to the Pareto frontier
 3. Repeat steps 1 and 2 until there is no change in the prompt after 3 iterations.

END_RUN_GEPA Repeat RUN_GEPA until there is no change in the prompt after 3 iterations.

At each step, keep track of the best prompt, which has the maximum number of successfully predicted data points, in `./pareto/optimal.md`. MAKE SURE THAT YOU DO NOT DO REWARD HACKING OR CHEATING IN THE AGENT YOU ARE IMPLEMENTING TO FIT THE DATA. YOUR SOLUTION MUST GENERALIZE BEYOND THE DATA PROVIDED. Use internet search extensively at every step. Do not worry about budget. Create an HTML report with diagrams and illustrations in `./reports` and open it in the user's default browser. Do NOT STOP until you cannot improve the accuracy and recall after three consecutive rollouts. Use gpt-5.6-sol model (not codex) for a thorough review of the work done at every step by the other model.

What the prompt formulation adds. Beyond the shared clauses, expressing GEPA as a chat prompt, where the reference implementation is a Python library, has three template-specific consequences. There is no optimizer code: the user does not write or maintain a Pareto-frontier data structure, a minibatch sampler, or a reflection-prompt template, as a user of OPRO, MIPROv2, ProTeGi, TextGrad, or the reference GEPA implementation does; the agent keeps the frontier as a flat `./pareto/` directory tree (one folder per node containing `prompt.md` and `score.json`) that the user can inspect and version-control. The persistence layer is the trajectory store: the target ChatSorcarAgent writes every task, its cost, and its result to `~/kiss/sorcar.db`, and every tool call and model response when the task runs in a chat webview (Section 2.4), so the GEPA reflector can re-read a past trajectory at the tool call that produced the failure, which is the trajectory-level reflection the GEPA paper credits for its gains over scalar-reward RL, with no integration code. And the review is cross-vendor: the closing clause “Use gpt-5.6-sol model (not codex) for a thorough review of the work done at every step by the other model” makes a model from a different vendor audit the rewriter’s proposals, so that a reward-hacking edit has to pass a reviewer that did not write it.

A.4 Repository Optimization

Repository optimization is the problem of taking an existing codebase together with a runnable command (a build, a benchmark, a server, an inference script, or a training loop) and improving its observable metrics (speed, accuracy, recall, cost) by repeatedly editing the source, re-running the command, reading the output, and proposing the next edit [Yang et al., 2024b, Xia et al., 2024, Jimenez et al., 2024, Ouyang et al., 2025, Wang et al., 2025b, Gauthier, 2023, Anthropic, 2025b, Cheng et al., 2025]. This is structurally the same outer loop as AI discovery, but with two distinguishing features. First, the genotype is a *whole repository*, so the agent must reason about cross-file dependencies, build artifacts, and dynamic runtime behavior, with no fixed harness; this is the regime in which SWE-agent [Yang et al., 2024b] showed that a purpose-built agent-computer interface (the ACI) outperforms naive prompt-only baselines on SWE-bench, in which Agentless [Xia et al., 2024] showed that a fixed localize–repair–validate pipeline can rival agentic ones on SWE-bench Lite, and in which OpenHands [Wang et al., 2025b], Aider [Gauthier, 2023], and Claude Code [Anthropic, 2025b] ship repository-level coding interfaces as production tools. Second, the fitness function is a real running process (e.g. wall-clock latency, throughput, GPU memory, accuracy on a validation set, or cloud-dollar cost) so the agent must launch the command in the background, monitor its streaming output in real time, and abort speculative runs the moment a partial measurement renders them moot; KernelBench [Ouyang et al., 2025] shows that even frontier reasoning models initially beat the PyTorch baseline on fewer than 20% of GPU-kernel optimization workloads but improve

substantially once execution and profiler feedback is folded back into the prompt, and the ADRS thesis [Cheng et al., 2025] argues that this profile-edit-rerun loop is where LLM-driven research is currently most productive. The Sorcar Repository Optimization template instantiates this loop as a single chat prompt, with the Sorcar Agent’s streaming shell executor (Section 2.3) providing the real-time monitoring channel and the Worktree Sorcar Agent’s branch-per-task isolation (Section 2.5) providing reproducible rollback of every speculative edit.

The exact prompt shipped in `src/kiss/SAMPLE_TASKS.md` is reproduced verbatim below; the user fills in the command, the target folder or URL, the metrics of interest, and the concrete numeric targets:

Sorcar for Optimization: Can you run the command <<command>> and optimize it with respect to the following metrics: <<speed, accuracy, recall, cost>>. Then use AI discovery to optimize. You can add a diagnostic code that prints metrics, such as running time, at a finer granularity. Do not forget to remove the diagnostic code after the optimization is complete. You MUST NOT STOP until the metrics achieve the following values: <<give_concrete_values_for_metrics>>. Create a report.

The Bespoke OLAP task of Section 4.2 did not use this template. Its prompt was written for that project: the repository layout, the benchmark command, the baseline per-query times, a target of at most 783 ms on SF10, a researched two-round plan, six hard constraints, and process requirements, 5,979 characters in all.

What the prompt formulation adds. Beyond the shared clauses, the optimization template has three specific consequences. Measurement is real-time and cancellable: the Sorcar Agent’s streaming shell executor (Section 2.3) lets the agent launch the user’s command in the background, watch its output as it arrives, and cancel the run the moment a new edit invalidates it; without this, every speculative optimization would have to wait for a full benchmark to finish, the bottleneck KernelBench [Ouyang et al., 2025] identifies for GPU-kernel optimization. The task’s edits are isolated: they land on the task’s own `git` branch and worktree (Section 2.5), where the agent can commit each candidate and revert a regression discovered three iterations later with one `git` operation, without touching the user’s working tree, which gives a standard developer workflow the rollback that AlphaEvolve and OpenEvolve get from a programs database. And the agent instruments and then cleans up after itself: the clause “*You can add a diagnostic code that prints metrics. . . Do not forget to remove the diagnostic code after the optimization is complete*” lets the agent insert finer-grained profiling where its current hypothesis demands, and the system prompt’s Pre-Finish Verification rules (check each user requirement against what was delivered before calling `finish`) require the diagnostic code to be removed before the final commit.

Common pattern

All three templates share the same skeleton, and the shared clauses do most of the work. An *explicit numeric stopping condition* (“Do not STOP until. . .”) converts the open-ended search into a verifiable contract that the Relentless Agent’s continuation protocol (Section 2.2) keeps alive across context windows until the targets are met, with no human in the inner loop. A *held-out or generalization check* (mandated by the AI-discovery block of the system prompt for all three templates) and, in the GEPA prompt, an explicit anti-reward-hacking clause encode the generalization invariants that ADRS, ShinkaEvolve, and GEPA enforce through their evaluators, without harness code. A *journal file of tried ideas* (`./tmp/explored-ideas.md`, Algorithm 1) turns the file system into a long-term programs database analogous to FunSearch’s and AlphaEvolve’s, but inspectable and editable by the user; combined with the task’s own branch and worktree, every candidate can be committed and reverted. *Dynamic model switching* (`set_model`) lets a cheap model do routine bookkeeping and local edits while a frontier model is engaged only at the harder algorithmic decisions and review passes. The system prompt’s web-research protocol (ten distinct sites per research session, visited through the browser tool and logged to a counter file) couples the inner-loop measurement signal to the literature, so the agent’s improvements can come from the literature as well as from local micro-optimizations. Finally, a report written into `./reports/` makes every multi-hour run reviewable. The system prompt already encodes the engineering disciplines (read-before-modify, lint-and-test before `finish`, no shortcuts), so the user need only state the objective, the targets, and any external constraint (such as a GPU CLI to use).

B User-Facing Features

We release our system as three coordinated surfaces over a single local daemon: a VS Code extension, a browser/mobile web app, and a Python client API (`kiss.server.sorcar.run`). Section 2 and Appendix C cover the agent architecture and the system prompt; this section covers the user-facing features, compared with GitHub Copilot [GitHub, 2021], Cursor [Anysphere, Inc., 2026], Windsurf [Codeium, 2024], Devin [Wu, 2024], and Aider [Gauthier, 2023] where a comparison applies.

B.1 Multi-Model, Multi-Vendor Workflows

KISS Sorcar ships with a catalog of 689 models spanning nine provider categories: 106 OpenAI, 15 Anthropic, 20 Gemini/Google, 103 Together AI, 8 Z.AI (GLM family), 10 Moonshot AI (Kimi), 403 OpenRouter, 15 Claude Code CLI (exposed under the `cc/*` namespace), and 9 OpenAI Codex CLI (`codex/*` namespace) entries (counted at version 2026.9.25), of which 671 are generation-capable, 505 are function-calling-capable, 7 are embedding models, and 2 are the typed-decision models behind the `decide` tool of Section 2.3 (`src/kiss/core/models/MODEL_INFO.json`). A task may mix models simply by issuing prompts that reference a different model id; the framework swaps the backend per call without reinitializing the agent or the chat session. The picker also lists, under a Router group, the two bundled extension agents that register as models (`autorouter` and `bestrouter`, Section 2.3); with one selected, every task of the tab runs through that agent. Beyond bundled providers, the Settings panel’s custom endpoint and headers, and its Custom Models section (stored in `~/.kiss/MY_MODELS.json`), configure any OpenAI-compatible HTTP server (including local models served by, for example, `vLLM` or `llama.cpp`) which keeps the bring-your-own-key, local-first posture even for self-hosted backends. Because keys and prompts travel directly from the developer’s machine to the chosen provider, no Sorcar-operated intermediary observes the traffic.

B.2 Dynamic Steering of a Running Task

A common failure mode of long-running AI tasks is that the user notices a misunderstanding, an additional constraint, or a wrong direction while the agent is already executing. KISS Sorcar lets the user type a follow-up natural-language message at any moment during a live task, and that message is appended to the running conversation before the next model step, without halting the in-flight tool call, discarding the work done so far, or starting a new task from scratch.

Steering is uniform across all surfaces. In the VS Code sidebar, the running-task input bar accepts follow-up messages while the task runs. The browser and mobile surfaces expose the same input through the local daemon.

Dynamic steering is orthogonal to the continuation mechanism of Section 2.2 (which preserves progress across context-window and step-budget boundaries) and to worktree isolation (Section 2.5); a steered task can still be committed-and-merged or discarded as a whole branch. Because external backends such as Claude Code CLI and the Codex CLI are exposed as model providers and third-party agents are exposed through the same tool layer (Appendix B.6), the same user message can steer a local KISS Agent, a worktree-isolated Sorcar task, parallel sub-agents, or work delegated to a supported external backend. We compare KISS Sorcar’s dynamic steering with related in-flight control mechanisms (Claude Code Remote Control, GitHub Copilot cloud agent follow-ups, Cursor Cloud Agents, Codex queue-versus-steer mode, Aider AI! comments, and the LangChain/LangGraph human-in-the-loop middleware) in Section 6.1.

B.3 Real-Time Budget Accountability

AI coding assistants typically operate on a subscription model (Copilot, Cursor) or a per-seat pricing model (Devin, Windsurf), both of which obscure the per-task cost.

The chat clients display cost tracking for every task: total tokens, dollar cost, and step count are updated at every agent step, in a status bar above the transcript in the web app and in a Task Info view or drawer in the VS Code extension, and each tool-call panel shows its elapsed time. A Spend subpanel of the sidebar aggregates the task database into a calendar heatmap of daily cost and a per-model breakdown. We enforce per-session and per-task budget ceilings. If a task exceeds its

budget, the agent raises a hard error and stops accumulating charges. The KISS Agent also appends the current usage to every tool result so that the model knows its limits.

B.4 Integrated Browser Automation

The browser tools of Section 2.3 are available in every task whose “Use web tools” setting is on (the default) and whose caller kept the built-in toolset. The browser is headless by default; the agent calls `show_browser()` to open a visible Chromium window whenever a page needs the human (a login, a CAPTCHA, or a bot check), so the developer can watch or take over, and hides it again afterwards.

This enables, for example, verifying a deployed web application after a code change, filling out web forms as part of a testing workflow, scraping documentation to inform a code generation task, or interacting with web-based developer tools (CI dashboards, issue trackers) without leaving the editor.

B.5 Python Client API and @-Mentions

Every surface is a thin client of the same local daemon (`kiss-web`): the VS Code sidebar, the browser/mobile web app, and the Python client API speak one JSON wire protocol over a Unix-domain socket or a WebSocket. The Python API, `kiss.server.sorcar.run`, lets any process run a one-shot task on the daemon and block until it finishes (for up to an hour by default; the task keeps running on the daemon if the caller’s wait expires). Its keyword options mirror the chat interface: model selection (`model`), per-task budget caps (`max_budget`), working-directory pinning (`work_dir`), worktree isolation (`use_worktree`), automatic commit on task finish (`auto_commit`), custom endpoints and headers (`model_config`), and toggles for browser tools (`use_web_tools`), parallel sub-agents (`is_parallel`), persistent memory (`use_memory`), and pre-run task classification (`classify_tasks`). A `chat_id` handle continues a prior chat, so a scripted follow-up task sees the earlier tasks of that chat as context, and the returned record carries the task’s text, success flag, cost, tokens, steps, chat id, and persisted task id. Three options change the agent itself. A caller-supplied Python file of extra tools (`tools`) is imported by the daemon, and the functions returned by its `get_tools()` become agent tools for the task; with `append_basic_tools=False` they and `finish` are the only tools. `system_prompt` replaces `SYSTEM.md` for the run and its sub-agents, and `append_to_system_prompt` and `append_to_prompt` add text to the system prompt or the task prompt instead. A *Sorcar Extension Agent* (`extension_agent_path`) is a Python file whose top-level functions are named after `run`’s parameters (`prompt()`, `model()`, `max_budget()`, `system_prompt()`, `tools()`, `tool_profile()`, `docker_image()`, ...; the transport and parent-task parameters have no getter) plus a mandatory `description()`; the daemon imports the file on every run and calls each function it defines to compute that parameter, and two further functions may return an `llm_call_hook` that rewrites the new messages before every model call and a `tool_call_hook` that can veto any tool call. One such file is a complete custom agent, with its own prompt, model, budget, tools, and safety hooks, that runs on the same daemon, appears in the same tab bar and history, and is accounted for like any other task. A file named `xxx/xxx_sea.py` is also the chat command `/xxx` (Appendix B.6).

The chat clients also offer file and folder *@-mentions* with ranked project-file completion, which let the user paste path-aware context into the prompt without leaving the keyboard.

B.6 Extensibility: MCP, Skills, and Third-Party Agents

KISS Sorcar exposes six extension points that let users add capabilities without forking the framework; the first five take code or configuration.

Model-Context-Protocol (MCP) servers. MCP servers are declared in either user scope (`~/.kiss/mcp.json`) or project scope (`<project>/.kiss/mcp.json`, with backward compatibility for `<project>/.mcp.json`), and the framework discovers them at agent startup. Servers may use `stdio`, `HTTP`, or `SSE` transports. A remote server that follows the MCP authorization specification (Notion, Linear, Asana, or any URL) is signed into from the chat: the agent’s `connect_mcp_server` tool registers the client with the server’s authorization server (by client-ID metadata document or dynamic client registration, with PKCE and a fixed loopback redirect) and returns the consent URL, the user clicks Allow in their own browser, and `finish_mcp_server_connect` stores the tokens under `~/.kiss/mcp_auth/`, where later runs refresh them without asking again. Discovered tools become first-class KISSAgent tools alongside the built-in Bash/Edit/Read/Write tools.

Agent Skills. At startup the framework scans `~/.kiss/skills`, `<project>/.kiss/skills`, Anthropic Claude skill directories, `.agents/skills`, and the bundled Sorcar skills directory. The skills are exposed through one `skill(name)` tool whose description lists each skill's name and one-line description; calling it loads that skill's `SKILL.md` into the context and lists its bundled scripts and reference files, which the agent reads on demand. The tool is registered only when at least one user or project skill exists; the bundled skills alone do not add it. This mirrors Anthropic's Claude SKILLS [Anthropic, 2025b] convention so that the same skill files work in both ecosystems.

Third-party messaging, service, and automation agents. KISS Sorcar ships with 45 third-party agents under `src/kiss/agents/third_party_agents`. Thirty-two are messaging-channel agents: BlueBubbles, DingTalk, Discord, Email (IMAP/SMTP), Feishu, Gmail, Google Chat, Home Assistant, iMessage, IRC, LINE, Matrix, Mattermost, Microsoft Teams, Nextcloud Talk, Nostr, ntfy, Phone Control, QQ, Signal, SimpleX, Slack, SMS, Synology Chat, Telegram, Tlon, Twitch, Webhook, WeCom, Weixin, WhatsApp, and Zalo. Ten are service agents that give the agent authenticated tools: Brave Search, Firecrawl, GitHub, Google Calendar, Google Docs, Google Drive, Google Sheets, Notion, Overleaf (through the user's browser session cookie, since Overleaf has no public API), and PostgreSQL. One answers `/ask` questions about a running task from its persisted events. Two are infrastructure agents: an A2A agent that exposes Sorcar over the agent-to-agent protocol and an OpenAI-compatible HTTP server that serves Sorcar behind an OpenAI-style API. A Govee smart-home CLI for IoT lights ships alongside them. In a chat task the user simply states the goal ("send 'running late' to Alice on WhatsApp", "list my open GitHub PRs") and the agent dispatches the matching agent through its `run_agent` tool; each agent also has a CLI entry point (`kiss-slack`, `kiss-gmail`, ...). Gateway-capable channels also work inbound: a one-shot poll tick, normally scheduled as a recurring cron job, drains new messages and runs each as a Sorcar task, with thread continuity across ticks, a delivery ledger, sender allow-lists, and an optional pairing handshake so only approved senders can drive the agent. Every service agent carries its own check and authenticate tools (and a finish tool where the sign-in waits on a browser consent), so a task can connect a service on the spot, and the Apps subpanel of the sidebar starts such a task when the user clicks a service that is not connected. Where a provider supports a browser grant the user signs in in their own browser and nothing is pasted back: the OAuth device-code flow for GitHub and Microsoft Teams, PKCE with a fixed loopback redirect for Slack and Discord, the OAuth device grant for Matrix, Login Flow v2 for Nextcloud, and, for the six Google Workspace agents, a Composio connect link, since Google grants Workspace scopes only to verified apps; the calls then go through Composio's proxy and no Google token is stored locally. Signal links like Signal Desktop: the agent renders a `signal-cli` QR code that the user scans from the phone. On Linux, credentials for 18 of the connectors are isolated behind a local auth daemon ("Muse auth"): the agent process holds only surrogate tokens that the daemon swaps for real ones at the network edge, and every routed request is host-allowlisted and classified read or write against an allow/deny/ask policy with an audit log. The agent never asks for a password or a two-factor code; on a login wall it hands the step to the user. This breadth distinguishes KISS Sorcar from Claude Code (whose documented channels include Slack, mobile remote control, and research-preview Telegram/Discord/iMessage but no built-in Gmail, WhatsApp, phone-call, or SMS) and from Cursor (whose Cloud Agent integrations cover Slack and Microsoft Teams).

Scheduled automations. Cron jobs are not a built-in tool; the agent dispatches them through `run_agent("cron", ...)`, which runs `kiss.agents.sorcar.cron_agent` as an agent script. A user asks in natural language ("every 2 minutes, run a gateway tick on the Slack channel sorcar, with pairing", "check my Gmail every hour and ping me on Slack if something needs my attention") and the cron agent creates, lists, pauses, resumes, removes, or immediately runs the corresponding job in `~/.kiss/cron/jobs.json`, which a scheduler thread of the `kiss-web` daemon ticks about once a minute; a due prompt job runs as a fresh Sorcar task, and a due command job runs its shell command directly, without a model. Schedule times are Pacific wall-clock time whatever the daemon machine's clock, and a Schedule subpanel of the sidebar lists the jobs.

Sorcar Extension Agents as slash commands. A folder `xxx/` that holds the script `xxx_sea.py` (Appendix B.5) is the chat command `/xxx`: a prompt that starts with it is rewritten so the agent calls `run_agent` with the script and the rest of the prompt as the task, and `/xxx help` prints the script's `description()` without running it. The registry is built from the bundled third-party agents (`/slack`, `/gmail`, ..., and `/ask`, which answers a question about the current task from a digest of its persisted events through three read-only tools), the folders listed one per line in

`~/kiss/SEAS.md` (later lines win; a watcher picks up edits within two seconds), and the 14 bundled agents of `src/kiss/agents/seas/`, in that precedence: `/sh` runs one shell command with the Bash tool alone; `/merge` resolves a conflicted merge (Section 2.5); `/task_update` reports what a running task has done so far; `/remember` and `/forget` add a standing instruction to, or remove it from, `~/kiss/SORCAR.md`, a file appended to the system prompt of every task; `/autorouter` and `/bestrouter` route tasks to models (Section 2.3); `/git_extract_knowledge` indexes a repository’s files and commits into a block store and writes the curated pages of its domain memory, then schedules a daily refresh; `/skillopt` optimizes the prompt text of a skill or an agent against an evaluation set; `/write_paper` and `/review_paper` write or review a research paper with build, consistency, and word-limit gates as tools; `/rsi7d` mines the last seven days of the bundled agents’ runs for mistakes and cost sinks and proposes, applies, and evaluates edits to their system prompts; `/coding` runs the task with the agent’s defaults, and the same file, given a running container and a trial configuration, is the harness the benchmark trial runners use to run the agent unattended; and `/dummy` runs a plain sub-agent, the default when `run_agent` names none. A script whose `register_as_model()` returns true also appears in the model picker.

The sixth extension point needs no code: the package ships twelve sample-task prompts in `src/kiss/SAMPLE_TASKS.md` (the drivers of Appendix A among them) that users paste into the input bar, and the input bar offers six bundled *promptlets*, reusable instruction fragments such as the pair-with-review clause of Section 2.3, inserted from a toolbar panel or completed as ghost text. Users add their own promptlets and model catalog entries in two plain-text files under `~/kiss/` (`MY_INJECTION.md`, `MY_MODELS.json`) that are seeded once and never overwritten; the second lets a self-hosted OpenAI-compatible endpoint appear in the model picker with its own pricing and context length.

C The System Prompt

The system prompt is a structured document that, by default, governs the agent’s behavior on every task that the pre-run classifier of Section 2.3 does not mark as simple; a caller of the Python API may replace it (Appendix B.5). It specifies engineering practices.

XML-tagged structure. The prompt is organized using XML tags that delimit each concern: `<identity>`, `<visibility_constraint>`, `<tool_rules>`, `<web_research>`, `<code_style>`, `<workflow>`, `<testing>`, `<pre_finish_verification>`, and `<sorcar_specific>`. All three major LLM providers (Anthropic [Anthropic, 2025a], OpenAI [OpenAI, 2025b], and Google [Google, 2025a]) recommend XML tags for structuring complex prompts: they create unambiguous section boundaries that models read as structure, reducing the chance that the model misinterprets an instruction from one section as applying to another.

Scope of this section. `SYSTEM.md` also fixes the agent’s identity and a rule-precedence order (safety first, then rules marked MANDATORY, NON-NEGOTIABLE, or CRITICAL, then the user’s explicit instructions, then everything else), the visibility constraint that the user sees only the HTML summary passed to `finish` and speech played by `talk`, tool-usage rules (`Write` for new files and `Edit` for small changes, `decide` for decisions, `run_parallel` nested at most two deep, command-specific synchronous Bash timeouts with detached background jobs for commands expected to run longer than a minute, `run_commands_parallel` for parallel shell commands, reviewer sub-agents dispatched read-only on the reviewer’s model, all scratch files under `./tmp/`), the periodic-summary and voice rules, the ten-site web-research protocol with its confidence gate, the read-before-modify rule, the AI-discovery, adversarial-testing, and adversarial-training blocks (Appendix A), deep-work and planning rules, and a three-item pre-finish checklist. A task that the pre-run classifier of Section 2.3 marks as simple runs on `SYSTEM_LITE.md` instead, which keeps a shortened identity block, the visibility constraint, the background-command rule, the voice rule, and most of the repository-specific block, and none of the others. We describe only the two blocks that shape the code the agent writes: the coding instructions and the testing instructions. Both are reproduced verbatim from version 2026.9.25.

The read-before-modify rule of the `<workflow>` block, which the ablation of Section 3.2 removes in one arm, reads:

```
Read before modify rule - NON-NEGOTIABLE: You MUST call
Read(file_path) on every existing file BEFORE calling
```

```
Edit(file_path) on it or overwriting it with Write(file_path).
Never modify a file you have not Read in the current session.
Viewing a file through Bash ('cat', 'sed -n', 'grep', 'nl') does
NOT count: Edit and Write reject any existing file that was not
shown by the Read tool, and every rejected call wastes a full
step. Use Read (it deduplicates unchanged ranges, so it is
cheaper than 'sed -n') for any file you may later modify.
Exception: Write may overwrite a scratch file under a 'tmp/'
directory (e.g. ./tmp/PROGRESS.md, ./tmp/ideas.md) without a
prior Read; Edit still requires one.
```

C.1 Code Style Guidelines

The prompt encodes a minimalist code philosophy:

```
<code_style>

## Code Style

**MANDATORY, NON-NEGOTIABLE** Write simplest, clean,
readable, and minimal yet general code with minimal
indirection. These rules exist because over-abstracted
and complex code is harder to debug and maintain.

- Organize code across multiple files grouped by
  functionality.
- Prefer named functions, classes, and module-level
  helpers over closures and lambdas. Closures obscure
  control flow; use explicit parameter passing instead.
- Eliminate unnecessary attributes, locals, config vars,
  tight coupling, and attribute redirections.
- Eliminate redundant abstractions and duplicate code.
- Public methods must have full docstrings. Docstrings
  are part of the code, not "documentation".
- **MANDATORY (MUST FOLLOW): Fix root causes, not
  symptoms. Before writing code, ask: is the code SIMPLE
  and elegant?
- Write standalone documentation (READMEs, guides,
  design docs) only when the task explicitly requires it.

</code_style>
```

The block opens with two of the three markers that the prompt's rule-precedence order recognizes (the third is CRITICAL), so its rules rank above the user's explicit instructions and below only the safety constraints, and it states its reason in the second sentence, as Anthropic's prompting guide recommends [Anthropic, 2025a]. Each bullet names a habit we saw in generated code: helper classes and indirection where a function would do; new code piled onto whichever file is open; closures that capture mutable state and defeat type checkers; locals that hold a value for one line, constants moved to configuration files, and `self.x = other.x` redirections; utilities that duplicate an existing one; README and DESIGN.md files nobody asked for. The docstring bullet is also mechanical, because the KISS Agent builds each tool's schema from the docstring of the function that implements it (Section 2.1). The root-cause bullet is the only one with its own MANDATORY marker and the only one in bold: a user may ask for a quick patch, but the agent is still required to find the cause. Section 3.2 shows that on small tasks these rules change how the agent works and leave the hidden-test results unchanged.

C.2 Testing Instructions

The testing block reads:

```
<testing>

## Testing

- Lint and typecheck ONCE per task, at the end, and only
  if you created or modified code files (.py, .ts, .js,
  .css, .tsx, .jsx): run uv run check --full (or the
  project's equivalent) as part of Pre-Finish
  Verification, and fix every error in files you created
  or modified in this session (re-run it only to verify
```

```

those fixes). Do not run lint/typecheck during
development. uv run check --full runs every stage
(ruff, mypy, pyright, extension typecheck and lint)
and lists all failed stages at the end, so fix
everything it reports before the single verification
re-run instead of fixing one stage per run.
- Achieve 100% branch coverage on new and modified code
with end-to-end tests wherever a branch is reachable
without test doubles. If a branch is unreachable
without mocks (e.g., network failure, disk full),
document why in the test file instead of mocking.
- Write end-to-end tests only. **Do not write unit
tests** or use mocks, patches, fakes, or test doubles.
Each test must be independent and verify actual
behavior.
- DO NOT write structural tests which assert on the
source code.
- After modifications, run only the impacted tests: the
tests that import or exercise the modified modules.
Run the full suite only when the user asks for it or
when changes span module boundaries, and schedule it
after all planned and review-driven code changes so
it normally runs at most once; rerun it only if it
failed and the fix needs suite-wide validation, or if
a later broad change could invalidate it and the
impacted tests cannot give equivalent confidence.
- Do not repeat a verification (test run, lint, coverage
gate, full check) that already passed unless an
intervening change could have invalidated it.
- A failure that passes when re-run in isolation and
does not recur on one full re-run is a load-dependent
flake: name it in the final summary and move on;
never run the whole suite a third time to chase it.
- To confirm a suspected race condition: temporarily add
a random sleep (<0.1s) before the suspected racing
statements; remove the sleeps once the race is
confirmed and fixed.
- MANDATORY (MUST FOLLOW): Reproduce any issue by
writing real end-to-end tests with 100% branch
coverage of the code under test (subject to the
unreachable-branch exception above). Then fix the
issue. You can use screenshots to validate the
implementation. You MUST do the same for any feature
implementation.
- **MANDATORY (MUST FOLLOW)**: Before running all tests or
tests in a folder, split the set of tests equally by
the number of test methods into min(number of test
methods, max(1, cores - 2)) splits and run all splits
in parallel using the run_commands_parallel tool
(one pytest command per split; skip splits that would
collect zero tests).

</testing>

```

The block is the most opinionated part of the prompt, and its rules have measurable traces in Section 3. *Lint once, at the end, on touched files*: an earlier wording, “fix all errors including pre-existing ones,” let a one-line fix grow into a repository-wide cleanup; the scope clause and the timing clause were added in turn, and a sentence telling the agent to list failures in untouched files was dropped between versions 2026.9.17 and 2026.9.18 (its commit message calls it a leniency note). *100% branch coverage, achieved rather than aimed for*, with the one exception (a branch unreachable without a test double) required to be documented in the test file. *End-to-end tests only, no mocks*: a suite of mocks can pass while the system is broken and couples the tests to production code [Spadini et al., 2017], and a mock-free unit test of a private helper tests the implementation and leaves the behavior the user asked for untested; in the task database, 11 of 1,376 agent-written test files import a mock library, none since June 2026 (Section 3.1), and the HydraKV suite of Section 4.1 (28 tests that drive log writes into a file-size limit and SIGKILL a checkpointing child) is what the rule produces at scale. *No structural tests* that assert on the shape of the source, because they survive nothing and prove nothing. *Run only the impacted tests, the full suite once*: this repository has 10,463 tests, and Goswami [2026] formalize the same selection problem; a flake that passes on one re-run is named and not chased. *Reproduce before fixing, for bugs and features alike*, so that the fix has a failing test behind it and the reviewer model has something to run; the screenshot clause is for the webview. *Sleeps to confirm a race*, removed once it is fixed. *Shard whole-suite runs*, $\min(\text{tests}, \max(1, \text{cores} - 2))$ ways, with the `number_of_cores` tool to evaluate the formula; an earlier wording sharded only suites of more

than 100 tests, and the June 2026 commit that removed the threshold made every whole-suite run parallel. Up to version 2026.9.18 the bullet sent each shard to a `run_parallel` sub-agent; it now sends one `pytest` command per shard to `run_commands_parallel`, which runs shell commands in threads with no model in the loop. The counts that follow are from the earlier wording. In the database the agent sharded 990 test runs, a median of 4 ways; in the ablation the bullet fired in 2 of 20 full-prompt cells, and removing it removed the sharding (Section 3.2).

D Painless Software Engineering with KISS Sorcar

A central claim of our system is that natural-language interaction can replace manual code inspection and ad hoc scripting for understanding and evolving nontrivial subsystems. While developing KISS Sorcar, we found two recurring workflows useful: *conversational redesign*, in which the developer asks the agent to explain a subsystem and then directs incremental changes in plain English; and *invariant-driven bug discovery and fixing*, in which the developer states a user-visible invariant the system **MUST** satisfy and asks the agent to reproduce any violation with an end-to-end test before fixing it. We illustrate both with real sessions from the project’s history, reproducing the prompts verbatim and summarizing the agent’s responses; both sessions predate the current task database on the development machine, so the first is checked against the commits of 9 April 2026 and the second is reported as the agent summarized it.

D.1 Conversational Redesign via Natural Language

In this workflow we first ask KISS Sorcar to generate a detailed, step-by-step description of a workflow or algorithm we found buggy, and then we ask KISS Sorcar to revise some of the buggy steps in natural language. We illustrate this with a real development session in which the `worktree merge` workflow (Section 2.5) was first understood and then redesigned entirely through conversational prompts. The session comprises four consecutive tasks; we reproduce the prompts verbatim and summarize the agent’s responses.

Step 1: Understanding the existing workflow. The developer begins by asking the agent to explain the current post-task git lifecycle:

Can you tell me what happens, step by step, with git in `worktree_sorcar_agent.py` when a task finishes?

The agent reads the source code and returns a structured summary of the four-phase lifecycle: (1) during `run()`, a new branch and `worktree` are created and the task executes inside the `worktree`; when the task completes, *nothing is committed or merged*; the result is returned with merge instructions appended and the `worktree` stays pending; (2) `merge()` calls `_finalize_worktree()`, which stages all changes, generates a commit message via the LLM, commits, removes the `worktree`, and prunes; `merge()` then checks out the original branch and runs `git merge`; (3) `discard()` removes the `worktree`, prunes, and deletes the task branch without checking out the original branch first; (4) in interactive use a prompt asks the user to merge, discard, or skip, the last leaving the `worktree` pending. The agent also notes a key design invariant: nothing auto-merges; auto-commit occurs only at merge/finalize time; all steps are idempotent; and state can be recovered from git on restart.

Step 2: Simplifying the workflow via natural language. Armed with the workflow description, the developer decides the three-way choice (auto-merge, manual merge, discard) is unnecessarily complex and issues a redesign request:

Can you change `worktree_sorcar_agent.py` and the extension so that after the agent finishes its task, it simply asks "Commit and Merge" or "Discard". When "Commit and Merge" is clicked by the user, the agent must commit the changes with a generated commit message, merge the branch with the original branch, and delete the `worktree` and the branch associated with the `worktree`. If the user clicks "discard", it must delete the `worktree` and the branch, and checkout the original branch.

The agent modifies five source files across Python, TypeScript, and JavaScript: it updates `discard()` to check out the original branch before deleting the task branch, removes the `manual_merge()`

method entirely, simplifies `merge_instructions()` to show only two options, updates the webview UI to replace the three-button toolbar with a two-button “Commit and Merge or Discard?” bar, removes the manual action type from the TypeScript type definitions, and removes the corresponding handler from the Python backend. The commit that landed this change (9 April 2026) removed the tests for the deleted manual-merge path (70 lines across two test files) and updated one routing test; the two worktree test files then held 19 tests, all passing.

Step 3: Investigating unexpected post-merge state. After testing the simplified flow, the developer notices that files appear in the Source Control panel of VS Code on the original branch after pressing “Commit and Merge” and asks:

After the user presses "Commit and Merge", why do the modified files show up as committed in the original branch?

The agent traces the exact execution path through `squash_merge_branch()` and discovers that the implementation deliberately unstages the squash-merged changes via `git reset HEAD` so they appear as uncommitted working-tree modifications in VS Code’s Source Control panel. The design intention was to let the user review the diff before committing manually. The agent reports this finding along with the relevant code snippet and a verified end state showing no new commits on the branch.

Step 4: Directing a design change in one sentence. The developer realizes that code review already happened while the worktree was active and decides the unstage step is unnecessary:

The review is already happening in the worktree branch. You don’t need the user to review the modified and new files in the original branch. Fix it.

The agent replaces the `git reset HEAD` call in `squash_merge_branch()` with a conditional `git commit -no-edit` that uses the auto-generated squash message. It adds a `git diff -cached -quiet` guard for the edge case where the merge produces no changes. One existing test (`test_merge_leaves_changes_uncommitted`) is renamed to `test_merge_commits_changes` and its assertions are updated to verify a clean working tree. Between Steps 2 and 4 the session had also added a 33-test `test_worktree_extension_workflow.py` (fifteen commits, all on 9 April 2026), so the worktree suites had grown from 19 to 52 tests, all passing after this commit.

Discussion. The four-step session illustrates the development loop that our system supports. The developer never opens a source file, never writes a line of code, and never runs a test manually. Instead, the entire cycle (understand the workflow, redesign it, investigate an anomaly, direct a fix) happens through natural-language prompts, with the agent handling code reading, multi-file editing, test updates, and verification. This style of development becomes possible because of the agent hierarchy described in Section 2: the Worktree Sorcar Agent isolates changes on a branch, the Chat Sorcar Agent preserves conversational context across tasks, and the Relentless Agent automatically continues when the context window is exhausted.

D.2 Invariant-Driven Bug Discovery and Fixing

A second workflow we use frequently is *invariant-driven bug discovery and fixing*. The developer never localizes the defect or even names the suspect file. Instead, the prompt (i) states a user-visible invariant that the system MUST satisfy, (ii) asks the agent to reproduce any violation by writing an end-to-end test before touching the source, (iii) asks for the fix, and (iv) assigns one model to write the test and the fix and a second model from a different vendor to review and debug that work, explicitly asking the reviewer to look for missed call sites and newly introduced bugs.

We illustrate the pattern with a task in which two invariants of the input box of the interactive terminal client (a surface KISS Sorcar shipped at the time) were stated together:

in sorcar cli interactive, the user MUST be able to enter multi-line inputs in the input box. Moreover, all text shown must be word wrapped. Reproduce the issue by writing an integration test. Then fix the issue. Use claude-opus-4-7 model for all tasks including coding, bug fixing, and test creation. Use gpt-5.5 model (not codex) for thorough review and debugging of the work done by the

other model. Check if the other model has missed some code or has introduced bugs. No need to check if the models exist.

The agent executes the workflow in three phases.

Phase 1: Reproduce. Under `claude-opus-4-7` the agent locates the prompt loop in `src/kiss/agents/sorcar/cli_prompt.py`, writes a new end-to-end test file `test_cli_multiline_input.py` containing nine integration tests that drive a real `prompt_toolkit PromptSession` through `create_pipe_input` (the same pattern as the existing `test_at_mention_picker.py`), and confirms the first test fails: typing "hello", then Esc+Enter, then "world", then Enter returns just "hello" against the unmodified code, witnessing the multi-line invariant violation. The diagnosis is that `PromptSession` was instantiated without `multiline=True`, so Enter always submitted and the buffer never wrapped.

Phase 2: Fix. Still under `claude-opus-4-7`, the agent passes `multiline=True`, `wrap_lines=True`, and a `prompt_continuation` callable to the session; the continuation callable returns an ANSI string that repaints the cyan | left border on every wrapped or continuation row so the framed panel stays visually consistent. An Enter binding filtered by `~completion_is_selected` calls `event.current_buffer.validate_and_handle()` so plain Enter still submits, while escape, enter (Alt+Enter), c-j (Ctrl+J), and the kitty/foot/WezTerm CSI-u sequence ESC[13;2u (matched as a raw-char tuple because `prompt_toolkit` does not pre-map it) insert real newlines. The welcome banner in `cli_repl.py` is updated to document the new keys.

Phase 3: Cross-model review. The agent switches to `gpt-5.5` (not Codex) and re-reads the diff, the surrounding modules, and the new tests, looking specifically for missed call sites and broken edge cases. The review surfaces a real `prompt_toolkit` library limitation that `claude-opus-4-7` had failed to notice: the `xterm modifyOtherKeys Shift+Enter escape \x1b[27;2;13~` is pre-mapped to `Keys.ControlM` inside `prompt_toolkit.input.ansi_escape_sequences.ANSI_SEQUENCES`, which means any user binding for it can never fire. A previously-added (always-failing) test asserting that this sequence inserts a newline is replaced with one asserting the documented fall-through behavior (it submits like plain Enter), and the welcome banner is amended to direct users on those terminals to Alt+Enter or Ctrl+J. The review also corrects ruff E402/I001 import-ordering violations introduced during the fix. Verification runs `uv run pytest -v` on the new file (9 tests) together with the neighboring CLI suites `test_at_mention_picker.py`, `test_cli_repl.py`, and `test_cli_panel.py` (40 tests), 49 of 49 passing over the four files, followed by `uv run check -full` (ruff, mypy, mdformat, syntax, api-docs) before the agent reports completion.

Discussion. The pattern generalizes. Recent entries in `sorcar.db` apply the same five-line template (“*X MUST hold. Reproduce by an end-to-end test. Then fix. Use model A for code, model B for review.*”) to fix invariant violations in the then-shipped terminal client’s notification stream, in the routing of bash tool outputs to the Result panel, and in syntax-highlighting the output of the Read tool. Three properties make the loop effective. First, by phrasing the bug as an invariant with no stack trace, the developer transfers diagnostic effort to the agent without having to localize the defect in advance. Second, requiring an executable end-to-end test before the fix produces a reproducible artifact that the reviewer model can refute or confirm independently of the implementer’s narrative, and that survives in the test suite as a regression guard. Third, splitting code production and code review across two vendors with different training data catches mistakes that a single model would systematically miss, including library-internal assumptions, such as the `prompt_toolkit` pre-mapping above, that no amount of self-reflection by one model is likely to surface. Dynamic model switching (Section 2.3) makes both halves of this split a single `set_model()` call inside one continuous task.

E Ablation Tasks, Arms, and Probes

Tasks. Table 7 lists the ten tasks. Each lives in `papers/kissorcar/ablation/tasks/<name>/` as a `repo/` fixture (one module and a `tests/test_basic.py` of passing tests; six of the ten also carry a `README.md` and a `conf_test.py`), a `task.md` that states the change in one or two paragraphs with exact signatures and output formats, a `hidden_tests/test_hidden.py` the agent never sees, and a `reference/` solution. The authoring rules were: standard library only; a fixture of at most 120

lines; every hidden test fails on the fixture and passes on the reference; the task text must be sufficient for the hidden tests, so that a failure is a defect and not an ambiguity. Tasks were written by the agent itself from those rules and checked by us; one task’s basic tests contradicted its own specification (the `csvstats` fixture asserted an exact stdout that the new `skipped=N` line invalidates) and was corrected before any cell ran.

Table 7: The ten ablation tasks. *Fixture* is the module the agent edits, with its non-blank line count; *basic* and *hidden* are test counts.

Task	Fixture	Basic	Hidden	Change asked for
<code>csvstats_groupby</code>	<code>csvstats.py</code> (28)	3	8	<code>-group-by</code> for a CSV column-statistics CLI; skipped-row line; exit 2 on unknown column
<code>dupfinder</code>	<code>dupes.py</code> (28)	3	9	three keyword options for a duplicate-file finder (min size, symlink policy, unreadable files)
<code>http_retry</code>	<code>fetch.py</code> (35)	3	8	retries with backoff for 5xx and connection errors; injectable sleep; final-status semantics
<code>ini_parser_fix</code>	<code>iniparse.py</code> (26)	3	7	six invariants for an INI parser (inline comments, quotes, duplicate keys, DEFAULT, line numbers)
<code>kvstore_ttl</code>	<code>kvstore.py</code> (35)	3	9	per-key TTL with an injectable clock, purge on mutation, legacy file format preserved
<code>log_rotate</code>	<code>rotate.py</code> (23)	4	8	size-based rotation with numbered backups, exact byte semantics, <code>backups=0</code>
<code>relative_dates</code>	<code>reldate.py</code> (23)	3	9	weeks, hours, weekday names, case and whitespace rules, error text for a relative-date parser
<code>run_cmd_timeout</code>	<code>runner.py</code> (34)	3	9	a <code>Result</code> dataclass, no-raise semantics, timeout, stdin for a subprocess runner
<code>todo_sqlite</code>	<code>todo.py</code> (44)	3	9	<code>done</code> and listing commands with exit codes for a SQLite todo CLI
<code>token_bucket</code>	<code>ratelimit.py</code> (37)	3	9	<code>allow</code> and <code>wait_time</code> for a token bucket with an injectable clock

Arms. `arms.py` builds the system prompts at run time from frozen copies of the `SYSTEM.md` and `SYSTEM_LITE.md` of version 2026.9.18 kept in `ablation/prompts/`, so later edits to the shipped prompt cannot change the arms. Five of the six arms reported here differ in the system prompt and the review arm in a suffix appended to the task; the file also defines same-vendor and self-review arms and logged report variants that this paper does not report. The three “minus” arms remove, by exact string match, the read-before-modify sentence of the `<workflow>` block; the coverage bullet, replaced by “Achieve 100% branch coverage on new and modified code.” (which drops the no-mocks clause and, with it, the unreachable-branch exception), the “Write end-to-end tests only ... test doubles” sentences, and the words “real end-to-end” of the reproduce-first bullet in `<testing>`; and the parallel-sharding bullet of `<testing>`. The review arm appends the promptlet of Section 2.3 to the task text, with `claude-fable-5` as developer and `gpt-5.6-sol` as reviewer. A cell is one `run_one.py` process: it copies the fixture into a fresh git repository under `/tmp`, runs `SorcarAgent.run(model_name, prompt_template, work_dir, max_budget, base_system_prompt, web_tools=False, use_memory=False, tool_call_hook)`, wraps `ChatSorcarAgent.run` so that every sub-agent’s calls are recorded as well, records each top-level tool call (for `Edit`: whether the file had been read and how many times the old string occurs; for `Write`: whether the file existed and whether a test file imports a mock library; for `Bash` and `run_parallel`: whether they run `pytest`), then runs the hidden and basic tests and writes one JSON record. `run_all.py` schedules cells task by task with the arms interleaved so a partial grid stays balanced.

Probes. For every defect a reviewer reported in the review arm, `probes/<task>/probe_<k>.py` is a standalone script that takes a sandbox directory, imports or runs the module from it, exercises the reported case, and exits 1 if the defect is present. `probes/run_probes.py` runs each probe against every sandbox of the same task and against the reference solution and writes `probes/<task>/results.json`; Section 3.3 reports the totals. Two remarks in one `todo_sqlite` review that the reviewer itself labeled non-blocking were not probed (`probes/todo_sqlite/NOTES.md`): one describes behavior `task.md` requires, the other behavior it does not cover.

F Case-Study Prompts and Process Record

The adversarial-discovery task prompt of the HydraKV project (Section 4.1), verbatim except for its closing paragraph, which asks for extensive internet search, names `claude-fable-5` for development and `gpt-5.6-sol` for read-only review at no more than 20% of the task budget, and asks that the model names be used literally:

```
Can you perform adversarial AI Discovery for the task at ./KV_TASK.md so that the goals in the task are met? Look at the previous task on how to validate the engine on the server. You MUST not stop until the goals are met. Generate adversarial workload variants to make sure that the engine works fast on the variant workloads. Do the following iteratively while maintaining a variable iteration_count variable which starts at 1 and increments by 1 on each iteration:
Generate a variant workload that are realistic like the original workload, but breaks the performance gain of the engine. Do extensive internet search to understand how to make the variant workload realistic to real-world workloads and robust to reward hacking or cheating. Then run the engine on the variant workload. If the goals are not met, use AI discovery to improve the engine on all workloads until you achieve the goals without cheating using the workloads. Then generate a new workload repeat the process until the engine can achieve the goals on the new test workload on which AI discovery was not performed.
```

The HydraKV tasks ran under the system prompt’s earlier discovery instruction, the same paragraph in all 8:

```
Mandatory Instructions (MUST FOLLOW): You will be exploring, implementing, and evaluating novel ideas while doing AI discovery or auto research or software optimization. Analyze the data/information provided to you and search the internet extensively to propose the first few ideas. Implement and experiment with each of your proposals. Note down the ideas you used to achieve user-specified metrics in a file along with the values of metrics, so that you can use the file to avoid repeating ideas that have already been tried and/or failed. You can also use the file to combine ideas that have been successful in the past. MAKE SURE THAT YOU DO NOT DO REWARD HACKING OR CHEATING IN THE MODELS OR AGENTS YOU ARE IMPLEMENTING TO FIT DATA. YOUR SOLUTION MUST GENERALIZE BEYOND THE DATA PROVIDED. You MUST search the internet at every step to find new ideas. Use powerful models (if available), such as claude-fable-5, gpt-5.6-sol-xhigh, to explore diverse kinds of novel ideas.
```

The discovery block of the current `SYSTEM.md` (version 2026.9.25), drawn in Figure 3, reads:

```
Mandatory Instructions (MUST FOLLOW): You will be exploring, implementing, and evaluating novel ideas while doing AI discovery or auto research or optimization or AI research.
1. read + profile the data / tests / baseline, record baseline metrics
2. web-search for SOTA approaches, papers, repos, issues
3. write ideas and rationale in ./tmp/ideas.md
4. Pairwise judge the ideas to find a winner idea.
5. Implement -> run real end-to-end evaluation -> log idea, aspect of improvement, and metrics in ./tmp/explored-ideas.md
   if better: keep, and try composing with prior winners on different aspects
   if worse: mark as failed so it is never retried
6. search again for fresh ideas not explored before and based on previous experience and exclude ideas that have been explored in ./tmp/explored-ideas.md; go to step 4
7. stop when the user’s metric goal is met, with a held-out / generalization check to prove it is not overfit
```

The Bespoke OLAP task (Section 4.2) ran an earlier wording of the seven steps of Algorithm 1 (“record baseline metric” and “go to 4” without “not explored before”), with a one-sentence adversarial-testing paragraph.

Table 8: HydraKV under the 7 workload mixes of the hardening task (`WORKLOAD_HARDENING.md`), 250 million keys each. Throughput on the scored mix is the July 21 reference-node re-run; the other mixes were run for correctness without tuning, and their throughput is reported as measured. “Task prompt” marks the two mixes quoted from the user feedback in the task’s prompt.

Workload mix	Proposed by	Mops/s	Integrity auditor
A_50_50	scored	5.50	0 fail
RMW_100	agent	0.30	0 fail
B_95_5	agent	2.93	0 fail
C_100_0	agent	2.78	0 fail
W_0_100	task prompt	5.09–5.13	0 fail
TIMESERIES_HD	agent	0.08	0 fail (1.28M deletes: 0 resurrections, 0 lost keys)
W95_5_WRITE	task prompt	5.17–5.24	0 fail

Table 9: The two case studies. Process rows come from the task database (development tasks identified by hand; hours are wall-clock; a continuation is one summary-and-restart sub-session). Results: HydraKV is the median of three cold-cache 30-second YCSB-A runs (50% reads, 50% blind upserts, Zipfian $\theta=0.95$, 250 million 64-bit keys with 100-byte values, 16 workers) on a GCP n2-standard-64 (64 vCPUs, eight local NVMe SSDs in RAID0) under an enforced 8 GiB resident-memory cap; the FASTER figure is the one stated in the task’s benchmark description (no FASTER run log exists). TPC-H is the sum of per-query median execution times over five repetitions on a 32-core Linux machine, validated against DuckDB at the paper’s tolerance (10^{-2}); seed 999 was held out.

		HydraKV	TPC-H engine
Process	top-level tasks; agent hours; model spend	8; 22.6; \$516	3; 2.4; \$90
	steps; longest task (hours); continuation sub-sessions	1,959; 5.8; 7	475; 1.3; 0
	steering messages; review sub-agents	11; 11	0; 1 (the first review ran as a separate task)
	kept idea groups; workloads tried; bugs fixed in the workload task (from workloads and tests / from the reviewer); e2e tests	5; 7; 8 (4 / 4); 28	4 change groups; 4 seeds; 2 review defects
Result	metric	YCSB-A Mops/s (higher is better)	SF10 seed 42 total, ms (lower is better)
	baseline	0.93 (FASTER, as stated)	3,132.2 (single-threaded artifact)
	agent’s engine	5.50	90.8
	other seeds (7 / 123 / 999 held out)		92.5 / 82.1 / 85.1
	SF1 seed 42, baseline to agent		276.8 to 14.2
	side cost	RSS 7.9 GB of 8 GiB	ingest 52.5 to 63.0 s (20%)